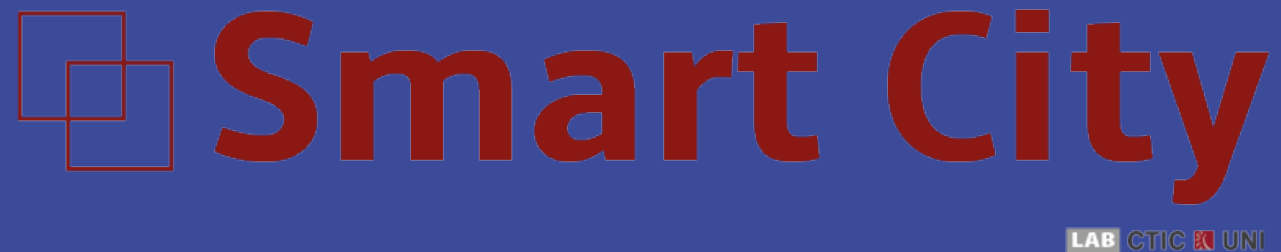
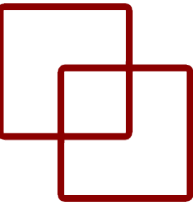


Redes Neurnoales Convolutionales



Dr. Manuel Castillo-Cara
Deep Learning con Keras y Python
Web: www.smartcityperu.org

Índice

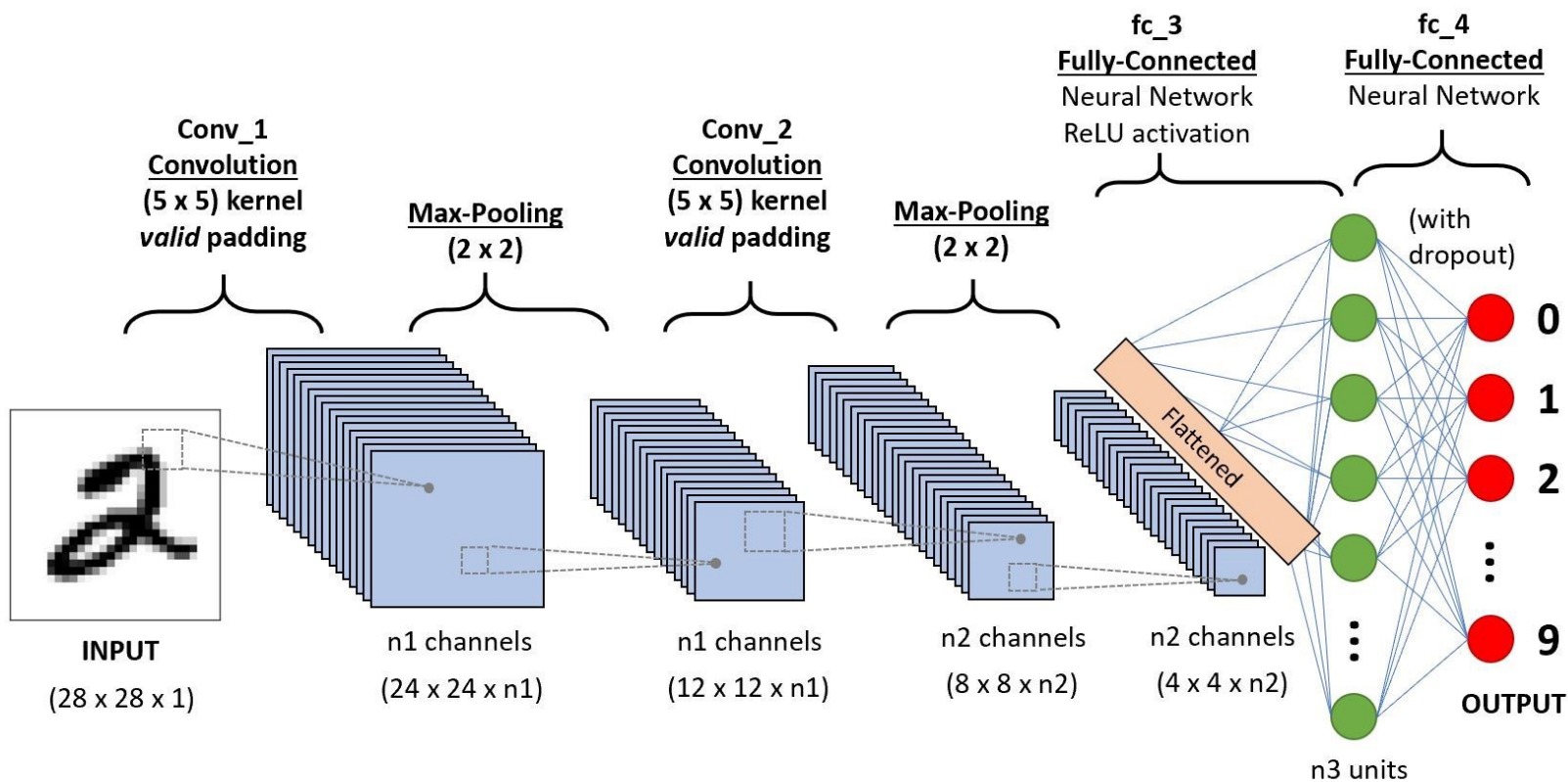
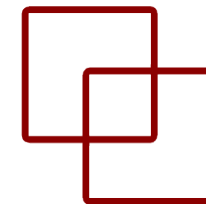


- Background
- Introducción
- Operación Convolución
- Operación *Pooling*
- Otros parámetros de la arquitectura CNN
- Arquitecturas utilizadas: casos de éxito
- Uso de GPUs para mejora de rendimiento
- Conclusiones

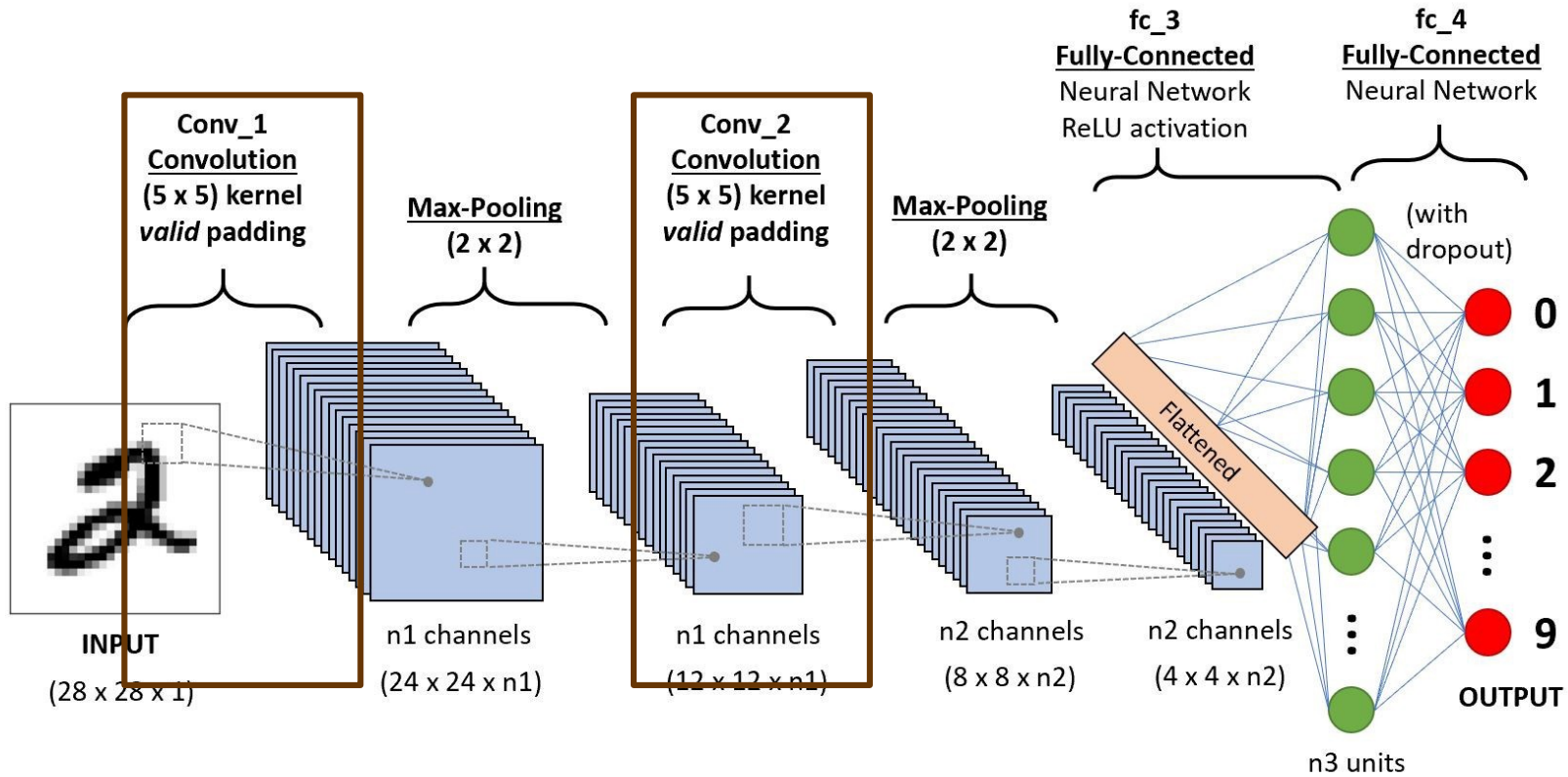


Background

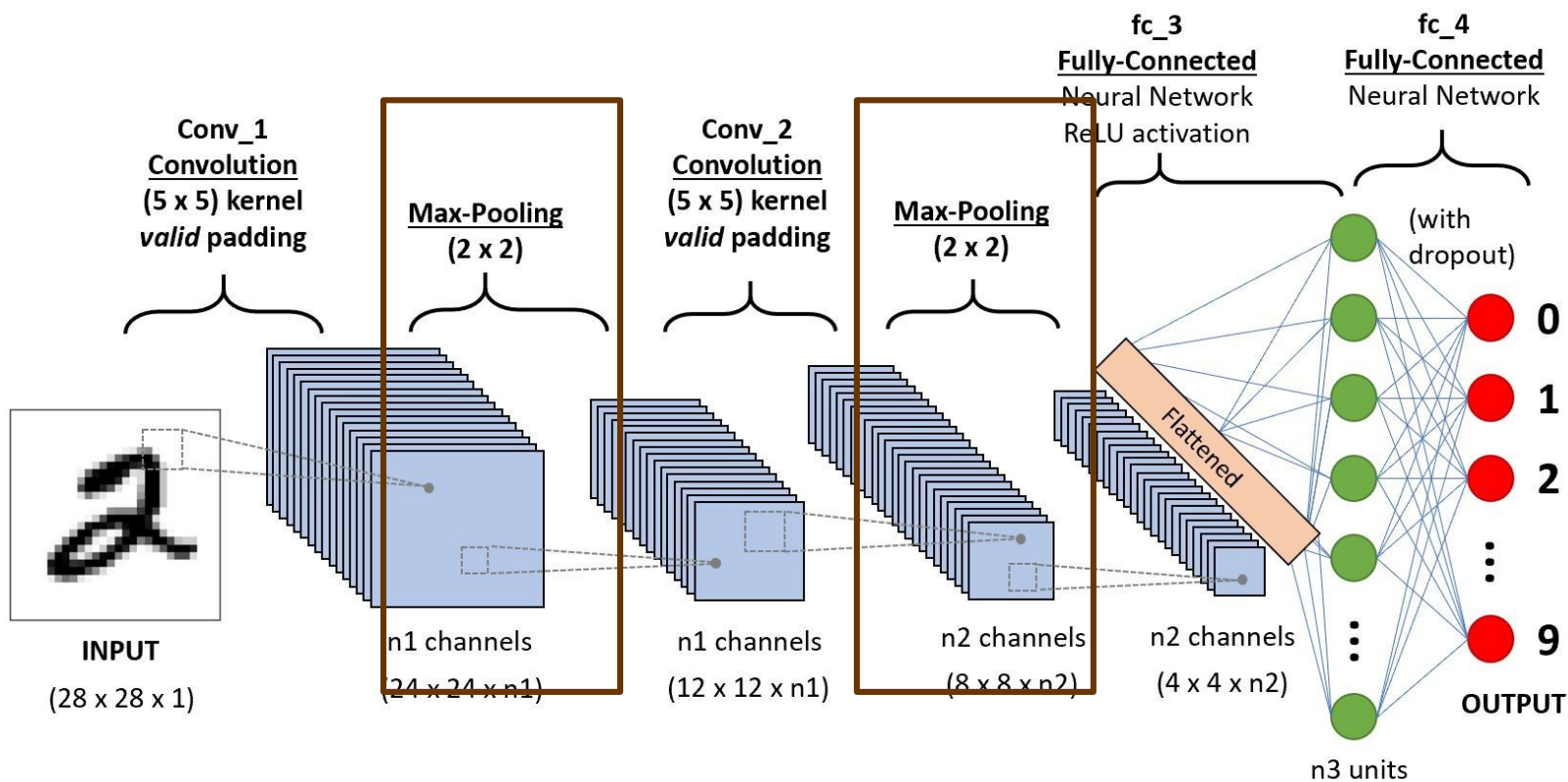
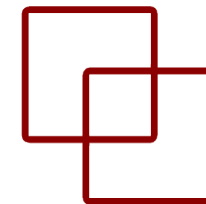
1. Background



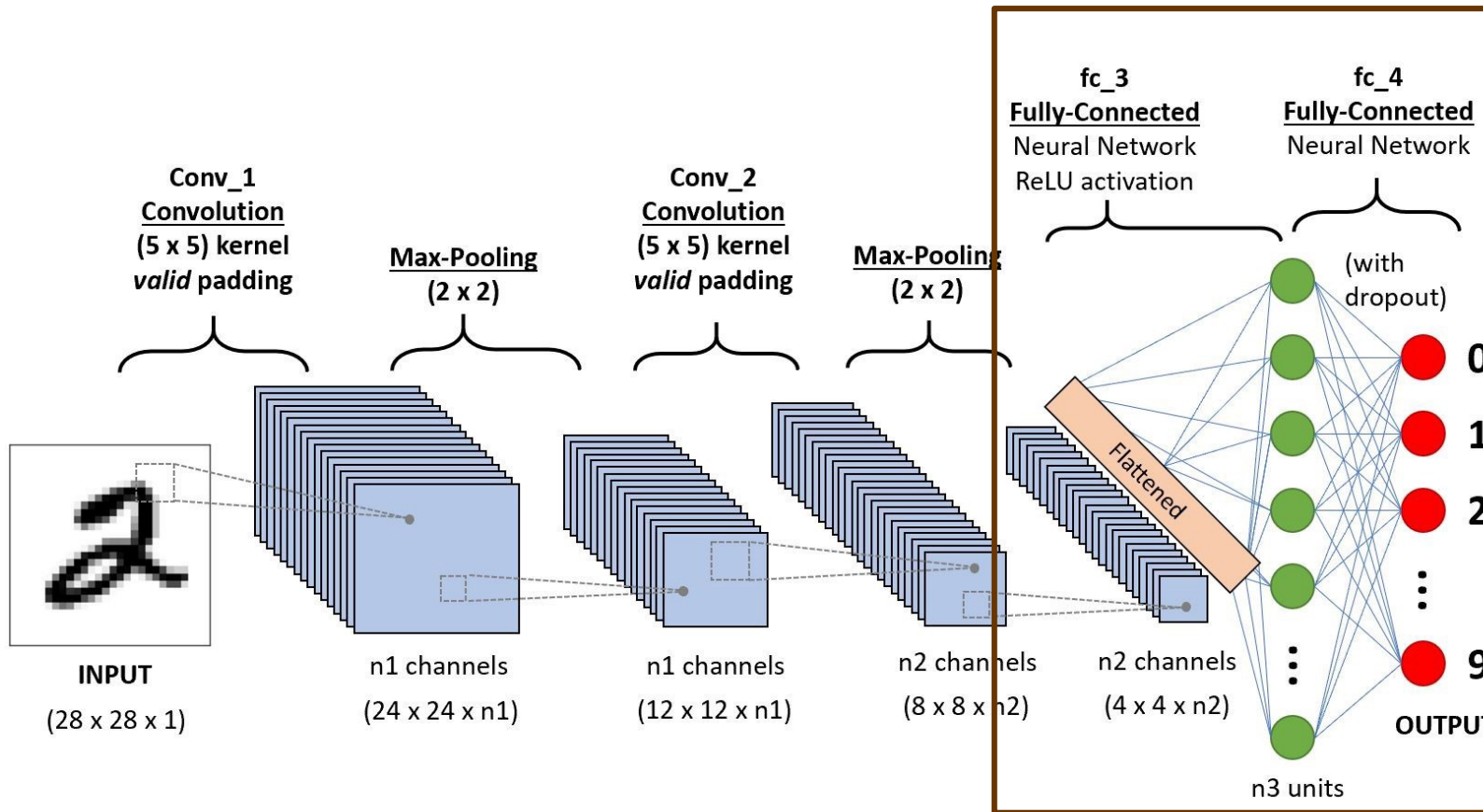
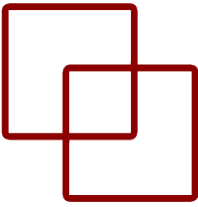
2. Capas convolucionales



3. Capas de agrupación (*pooling*)

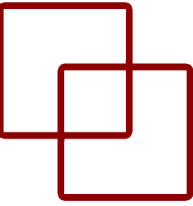


4. Capas completamente conectadas



5. Ejemplo

Imagen de entrada (32x32)

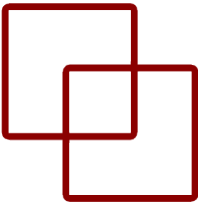


Input

**32x32x1
(B/N)**

5. Ejemplo

Imagen de entrada (32x32)



Input

**32x32x1
(B/N)**

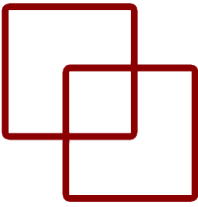
Convolution

**10 filtros
5px ancho
5px alto
ReLu**



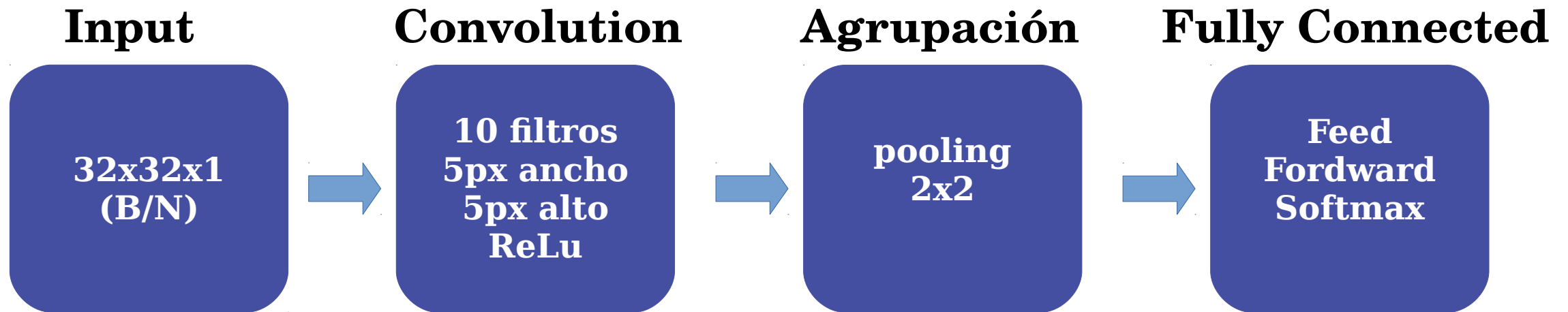
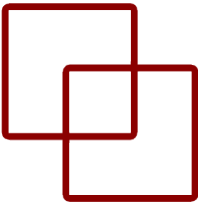
5. Ejemplo

Imagen de entrada (32x32)

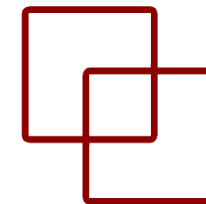


5. Ejemplo

Imagen de entrada (32x32)



6. Consejos



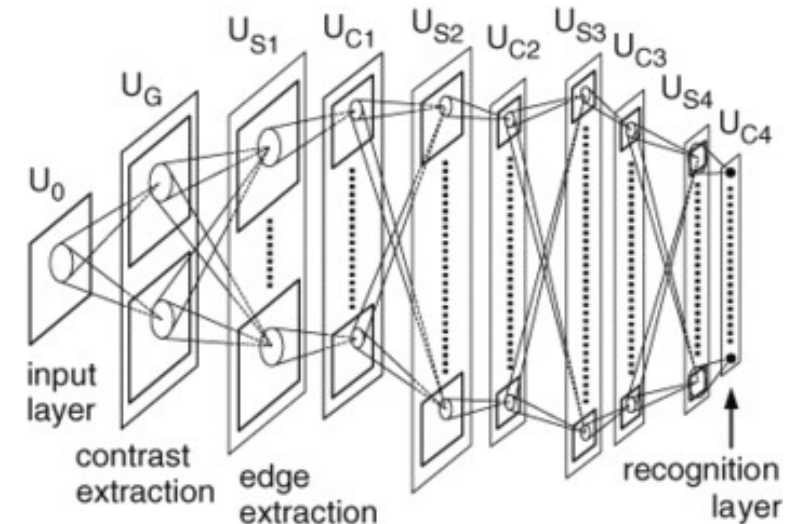
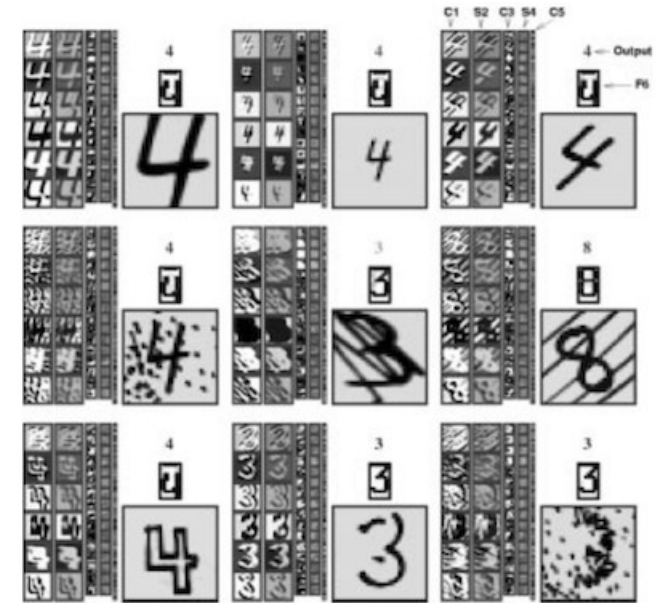
- Dimensiones del campo receptivo de entrada:
- Tamaño del campo receptivo
- Ancho de stride
- Número de filtros
- Relleno
- Pooling
- Preparación de datos
- Arquitectura de patrones
- Dropout



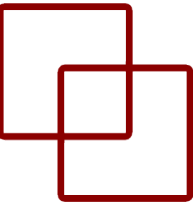
Introducción

1. Contexto (I)

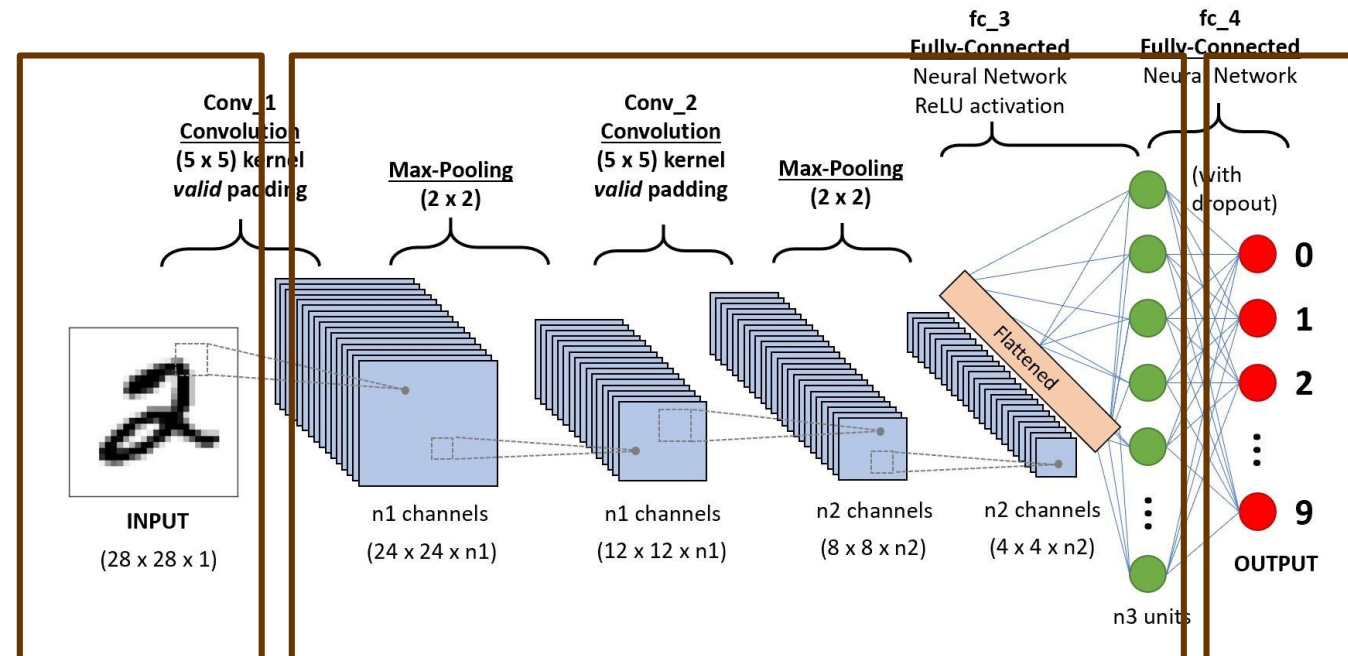
- Las redes convolucionales, comúnmente conocidas como CNNs (***Convolutional Neural Networks***) son un tipo específico de Red Neuronal (*Feed Forward*) basada en la **organización** de las neuronas de los cerebros biológicos
- Su comienzo se enmarca en un artículo llamado “*Gradient-based learning applied to document recognition*” y publicado en 1988 por **Yann LeCun**, el cual se basa asimismo en el artículo *Neocognitron* de **Kunihiko Fukushima** (1980)



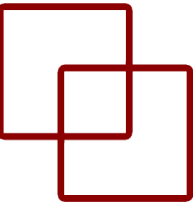
1. Contexto (II)



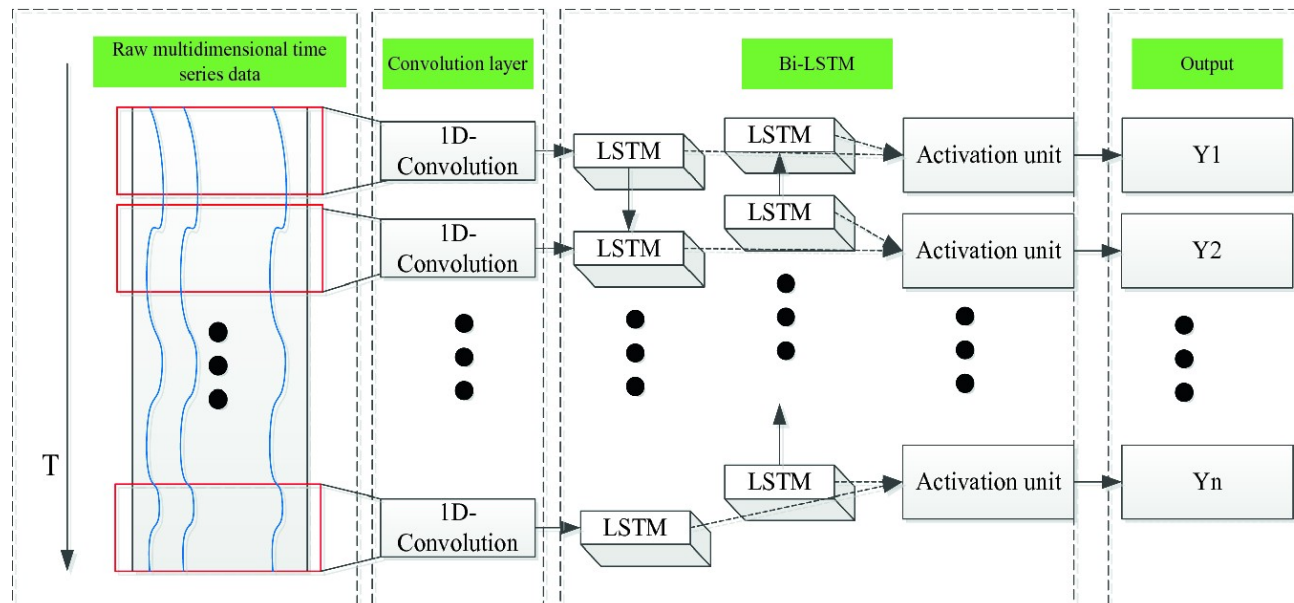
- Las redes convolucionales no dejan de ser redes neuronales, y por tanto incluyen una capa de **entrada**, una capa de **salida**, y diferentes **capas ocultas**
- Su especialización radica en las capas ocultas, las cuales se relacionan con **operaciones de convolución, max pooling** o *subsampling*, y capas totalmente conectadas
- Cada una de estas operaciones tiene un objetivo específico que se verá en esta unidad



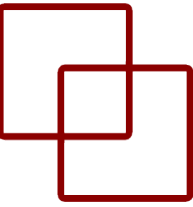
1. Contexto (III)



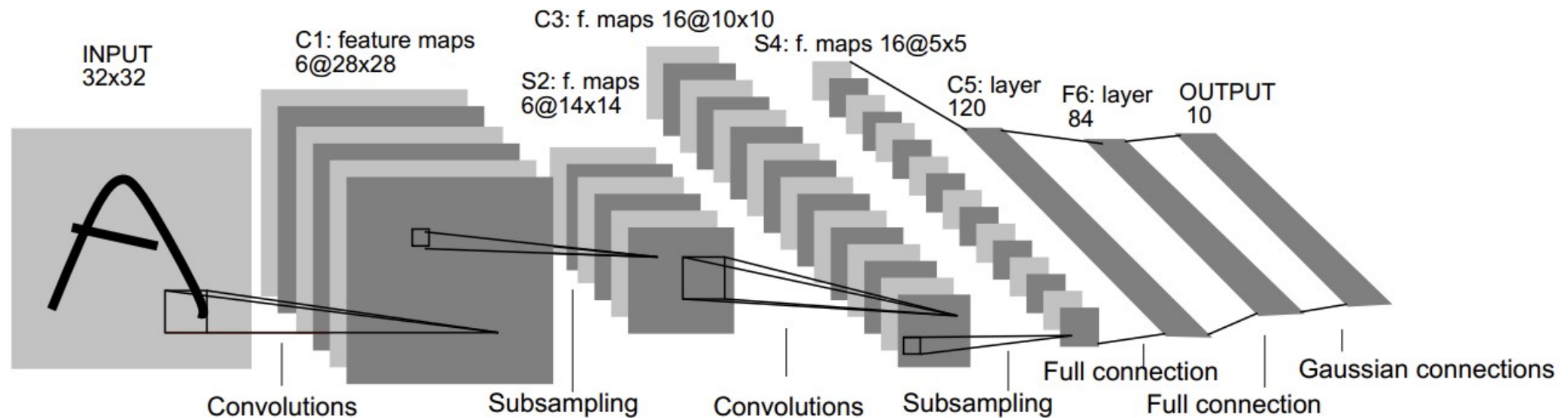
- A pesar de que la mayor parte de las aplicaciones de CNNs están relacionadas con la visión artificial, también se han usado en otros campos como las **series temporales**
- Su principal ventaja la encontramos a la hora de procesar datos que pueden representarse mediante **arrays**, como son las imágenes (arrays de 2 dimensiones en B/N) o series temporales con una única dimensión.



1. Contexto (IV)



- A continuación vemos el ejemplo más utilizado para presentar una arquitectura típica de CNN

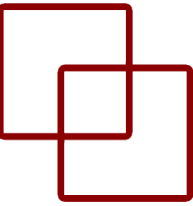


Fuente: Gradient-based learning applied to document recognition, Y. Lecun ; L. Bottou ; Y. Bengio ; P. Haffner

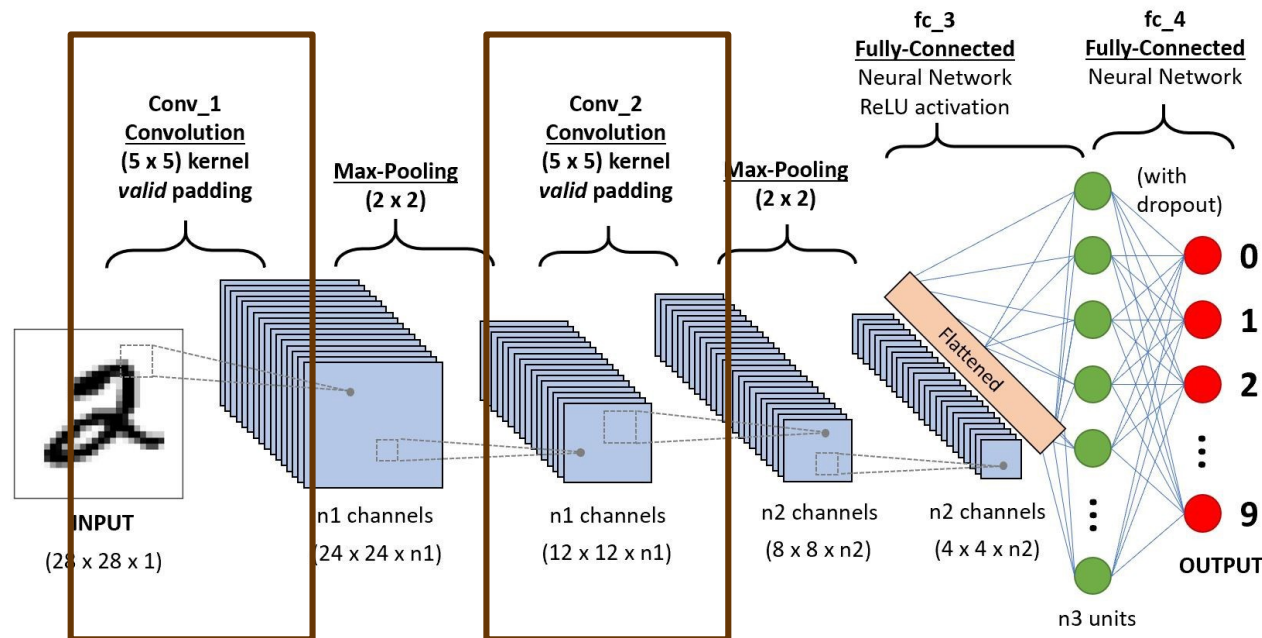


Operación Convolución

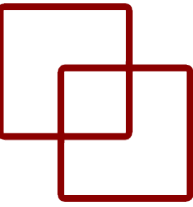
1. Operación convolución (I)



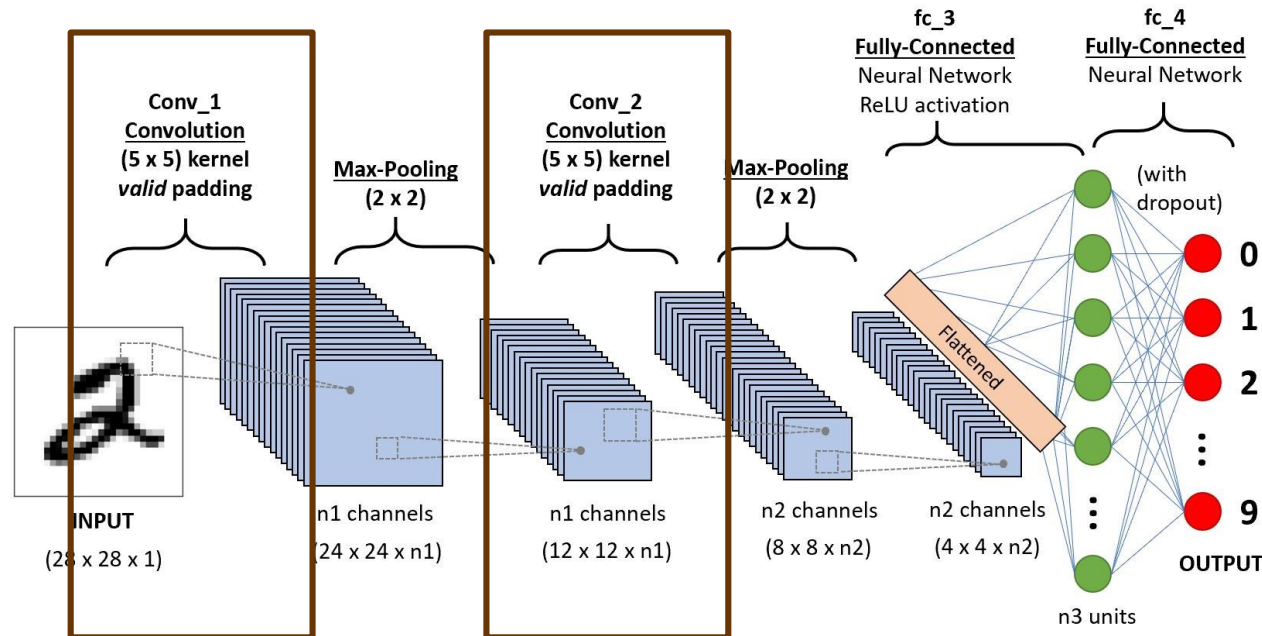
- La primera capa oculta de una CNN se suele corresponder con una **capa convolucional**, la cual puede entenderse como la codificación necesaria para la aplicación de filtros (conocidos como funciones *kernel*)
- Para una mejor comprensión de la operación convolución, pensemos en una capa de entrada que **codifica** una imagen binaria (B/N) y una convolución con la que queremos representar un filtro de **detección de bordes**.



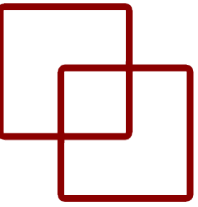
1. Operación convolución (II)



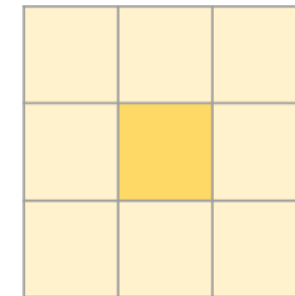
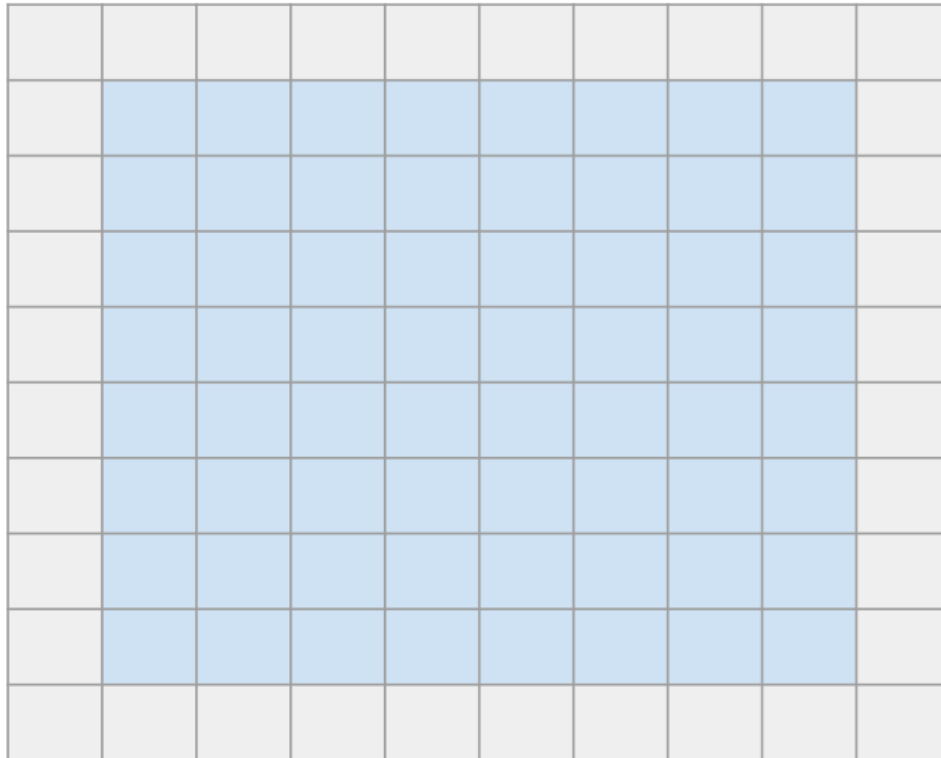
- Sin entrar en más detalles, la búsqueda de bordes se basa en buscar grandes diferencias entre un píxel y los píxeles que le rodean
- El **mismo tipo de operación** la queremos realizar sobre la imagen completa, por lo que aparece una primera cualidad: el uso de **pesos compartidos**
 - Esto significa que los parámetros / condiciones para detectar un borde en un pixel de la imagen deben ser iguales en todas las otras zonas de la imagen



2. Ejemplo



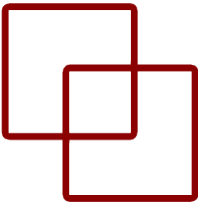
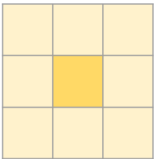
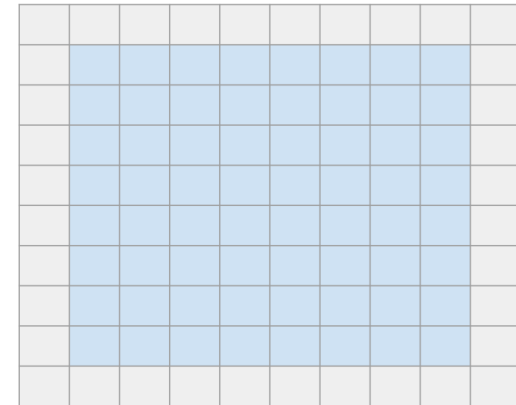
- Asumiendo una imagen de 10x10 píxeles, si la detección de un borde necesita trabajar con los 8 píxeles que le rodean tendríamos lo siguiente



2. Ejemplo

Operación convolución (I)

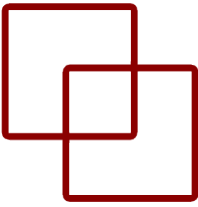
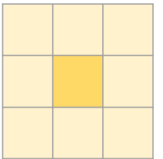
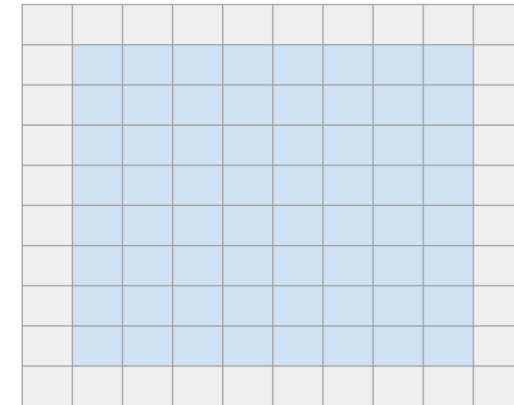
- En la imagen mostramos un filtro que se puede aplicar únicamente sobre las zonas resaltadas



2. Ejemplo

Operación convolución (I)

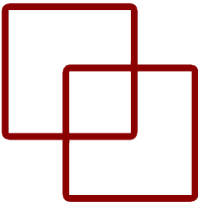
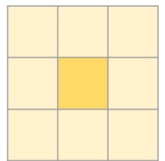
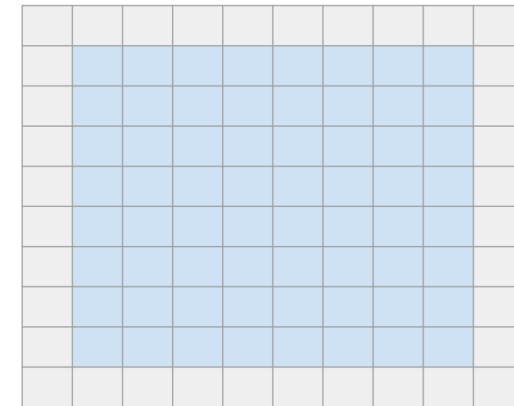
- En la imagen mostramos un filtro que se puede aplicar únicamente sobre las zonas resaltadas
- Trasladando el problema a resolver sobre la topología de una CNN, incluiríamos una capa inicial con 100 neuronas (puede mostrarse visualmente como una matriz de 10x10)



2. Ejemplo

Operación convolución (I)

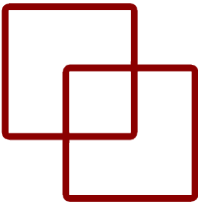
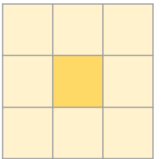
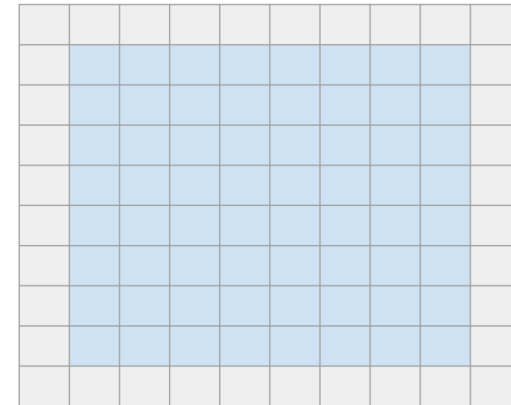
- En la imagen mostramos un filtro que se puede aplicar únicamente sobre las zonas resaltadas
- Trasladando el problema a resolver sobre la topología de una CNN, incluiríamos una capa inicial con 100 neuronas (puede mostrarse visualmente como una matriz de 10x10)
- La primera capa oculta contará con **una neurona por cada resultado** del filtro → tendrá 8x8 neuronas



2. Ejemplo

Operación convolución (II)

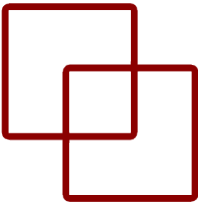
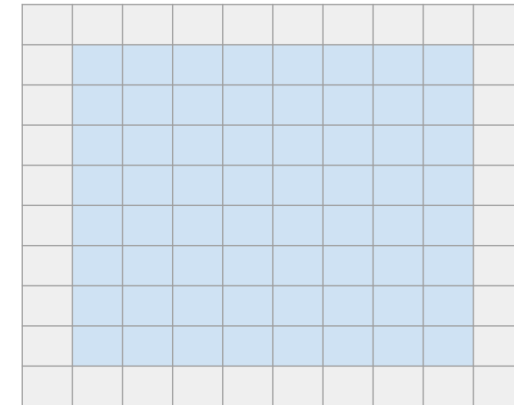
- Cada una de estas 64 neuronas estará conectada con 9 neuronas de la capa inicial, pero los **pesos** utilizados en esta interacción $w_1...w_9$ serán **compartidos** por todas las neuronas de esta capa oculta



2. Ejemplo

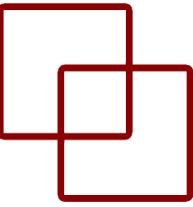
Operación convolución (II)

- Cada una de estas 64 neuronas estará conectada con 9 neuronas de la capa inicial, pero los **pesos** utilizados en esta interacción $w_1...w_9$ serán **compartidos** por todas las neuronas de esta capa oculta
- Esto **simplifica** el aprendizaje enormemente, además de dar **coherencia** al filtro a realizar

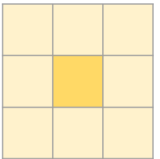
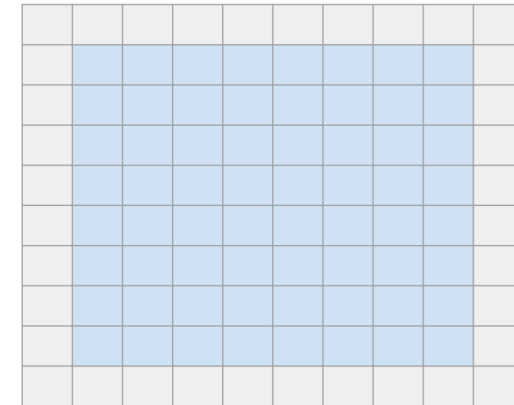


2. Ejemplo

Operación convolución (II)



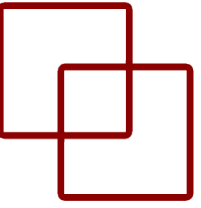
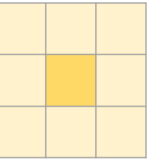
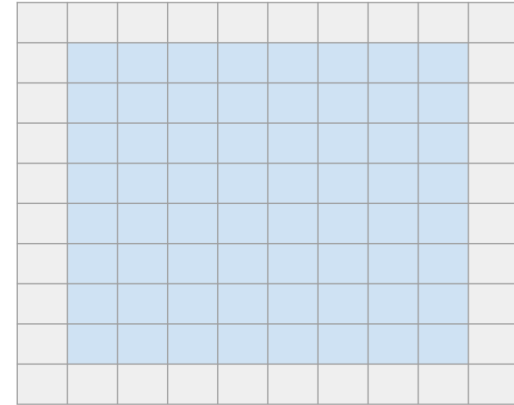
- Cada una de estas 64 neuronas estará conectada con 9 neuronas de la capa inicial, pero los **pesos** utilizados en esta interacción $w_1...w_9$ serán **compartidos** por todas las neuronas de esta capa oculta
- Esto **simplifica** el aprendizaje enormemente, además de dar **coherencia** al filtro a realizar
- Una vez aprendida, la función de activación (ReLU normalmente) nos permitirá saber qué píxel de la imagen se **corresponde o no con un borde**



2. Ejemplo

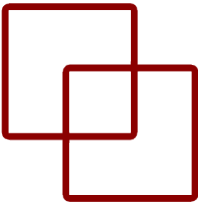
Operación convolución (III)

- En el ejemplo propuesto, se plantean convoluciones que involucren tanto al píxel bajo estudio, como los **8 píxeles a su alrededor** (3x3)

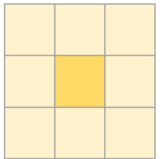
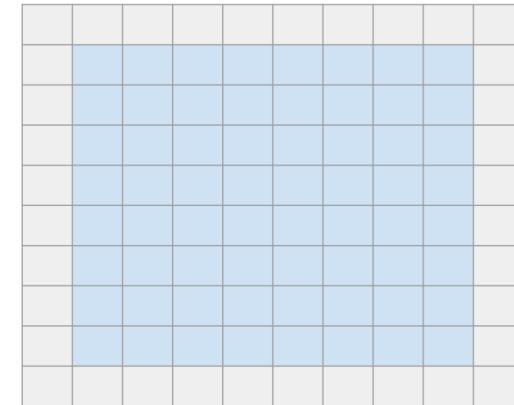


2. Ejemplo

Operación convolución (III)

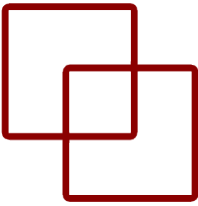


- En el ejemplo propuesto, se plantean convoluciones que involucren tanto al píxel bajo estudio, como los **8 píxeles a su alrededor** (3x3)
- Además, se asume que esta operación se realiza uno a uno, pero sólo sobre los píxeles de la imagen que es posible (salvo las filas / columnas 0 y 9), obteniendo como resultado una **capa de tamaño inferior** (10x10 → 8x8)
 - 2 parámetros definen esta configuración: ***padding*** y ***stride***



2. Ejemplo

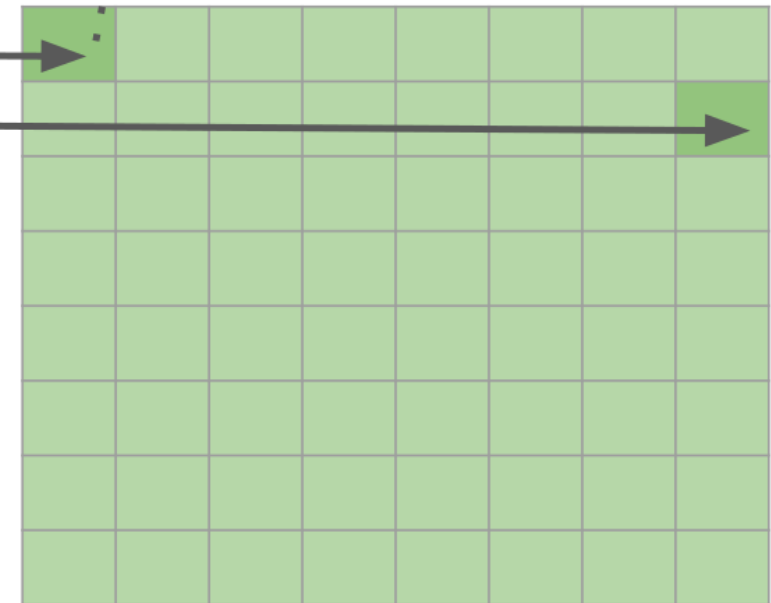
Operación convolución (IV)



1	3	2							
6	6	4							
5	7	6							

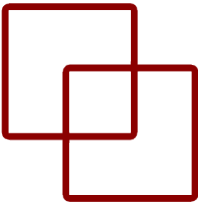
w_0	w_1	w_2
w_3	w_4	w_5
w_7	w_8	w_9

$$\begin{aligned} &= w_0 * 1 + w_1 * 3 + w_2 * 2 \\ &+ w_3 * 6 + w_4 * 6 + w_5 * 4 \\ &+ w_7 * 5 + w_8 * 7 + w_9 * 6 \end{aligned}$$

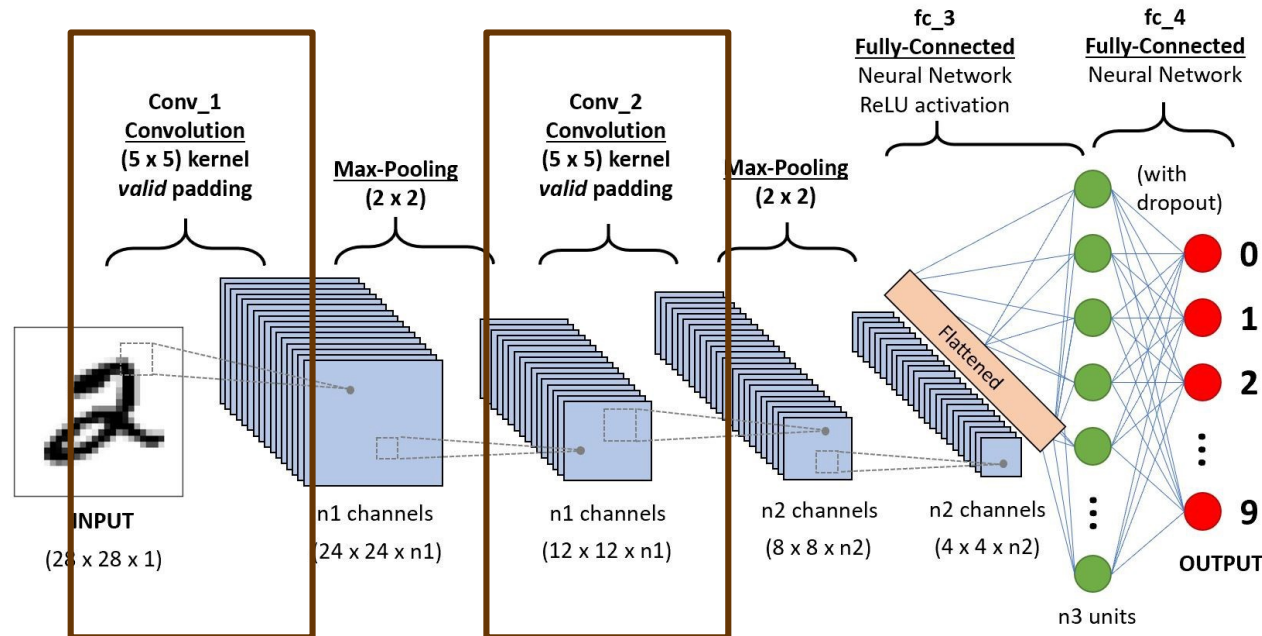


3. Padding

Definición

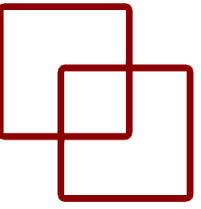


- El parámetro ***padding*** (opcional) define una serie de neuronas sintéticas usadas a modo de relleno, de forma que se permite realizar convoluciones sin que la capa resultado sea de un tamaño menor a la de entrada
- Estas neuronas deben tomar un **valor** para la realización de las operaciones, siendo cero el valor más usado.
 - En este caso nos referimos a este parámetro como *zero-padding N* (N = número de neuronas adicionales)

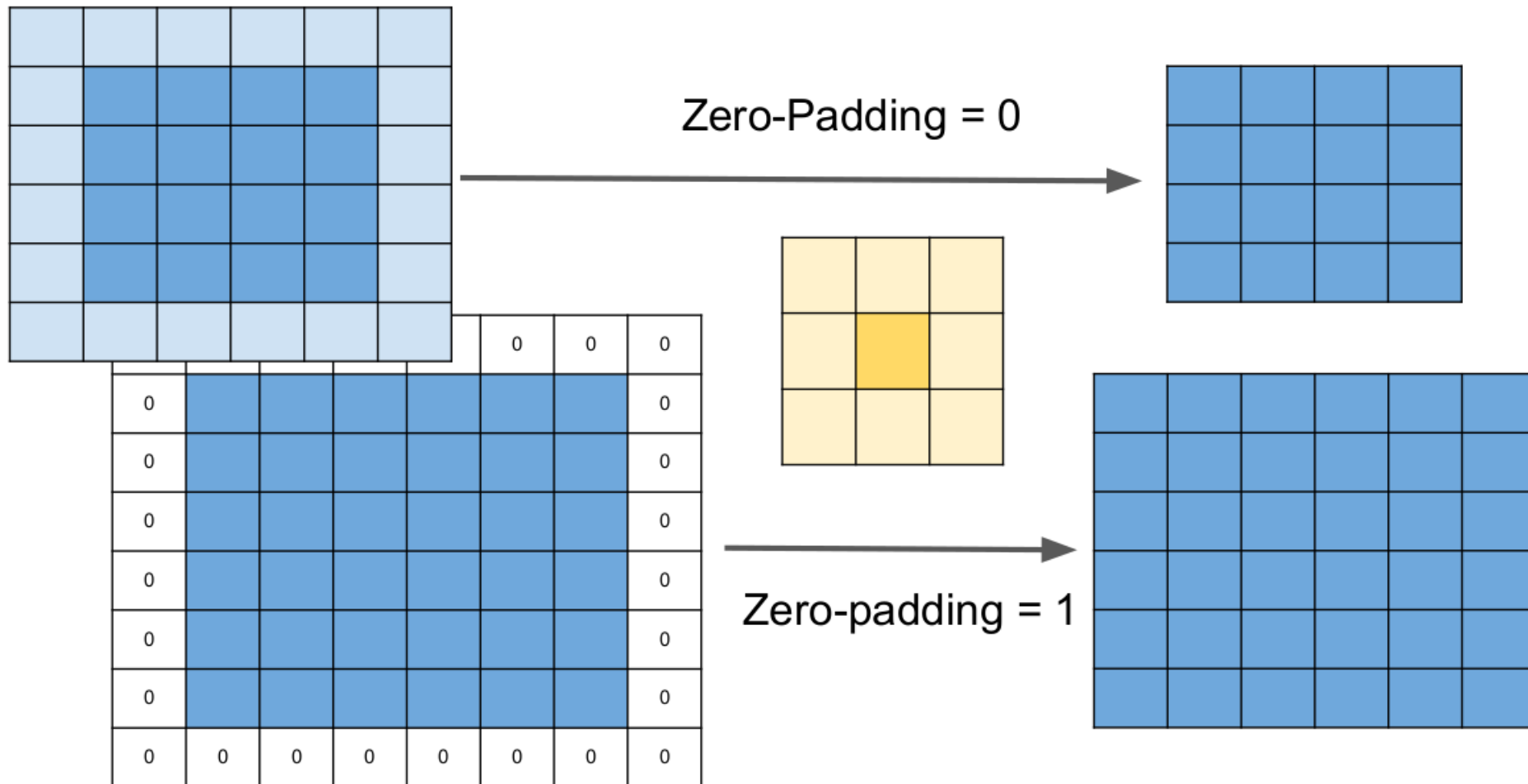


3. Padding

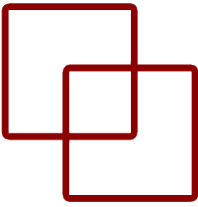
Ejemplo



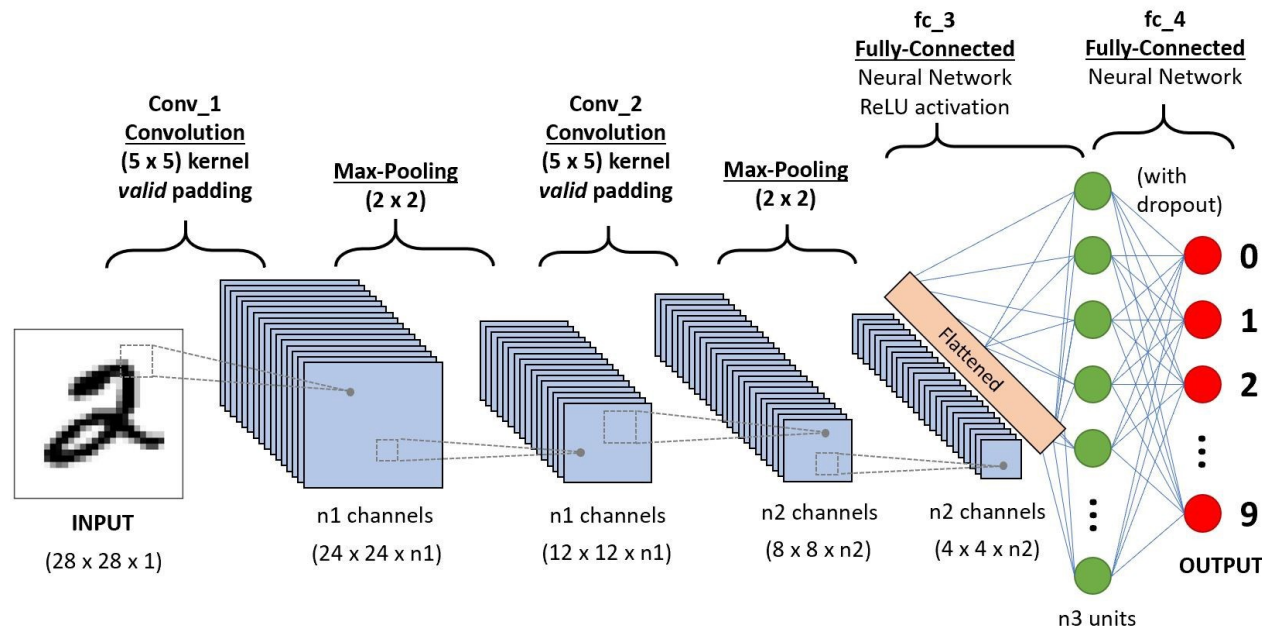
- Veamos un ejemplo con imágenes de 6x6 y un filtro de 3x3



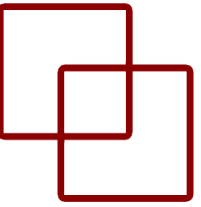
4. Stride Definición



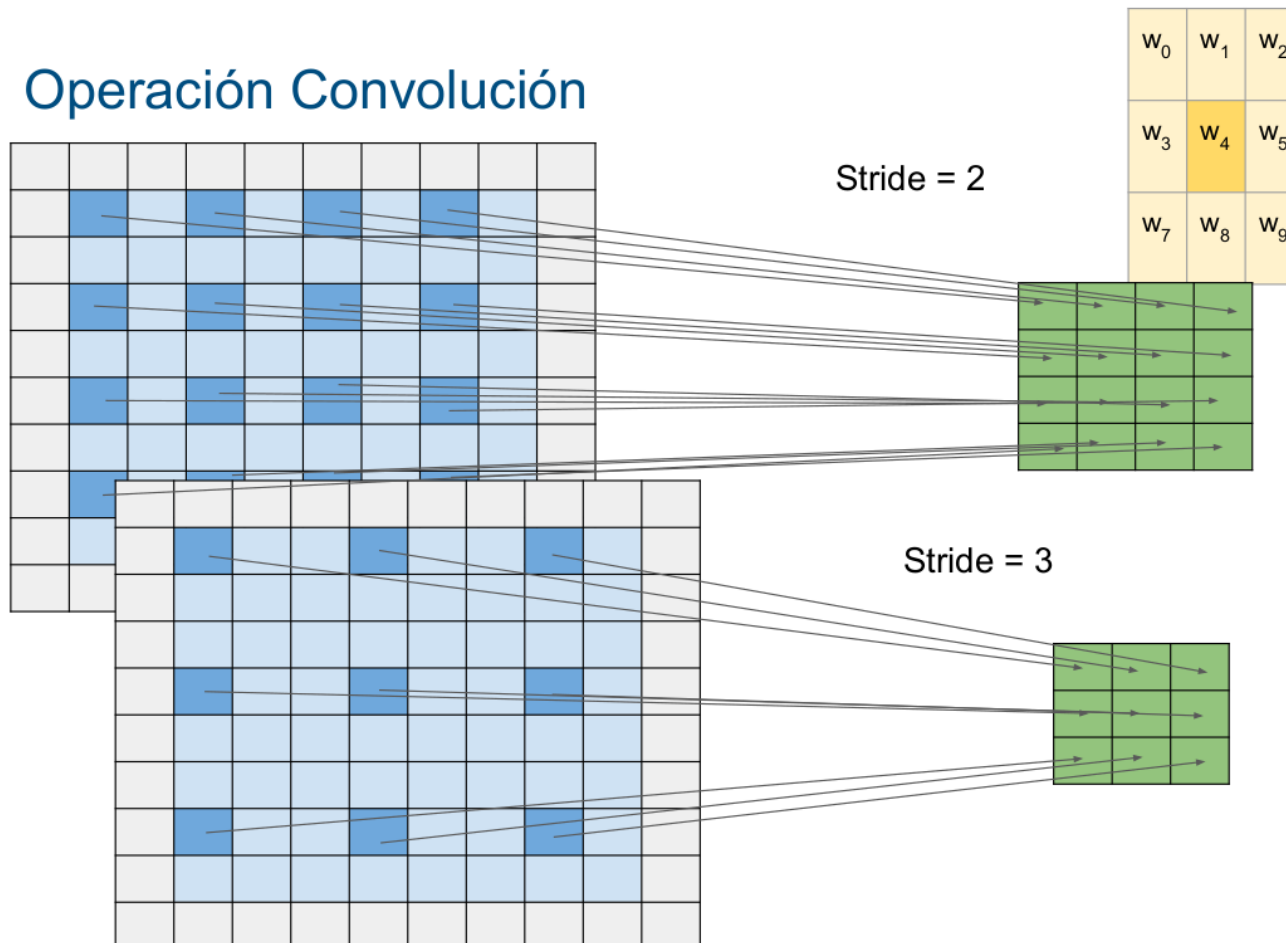
- El parámetro ***stride*** nos indica el número de neuronas a avanzar tras la aplicación de una convolución
- Para entender este parámetro recurrimos a la organización espacial: Usando una matriz de dos dimensiones, un valor de ***stride 2*** nos indicará que tras aplicar una convolución:
 - La siguiente se realizará **dos columnas a la derecha**, y
 - tras terminar la fila actual, la siguiente a procesar se seleccionará tras **bajar dos filas**



4. Stride Ejemplo

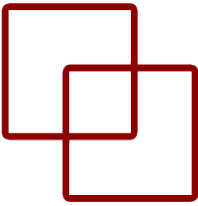


- Veamos un ejemplo con imágenes de 6x6 y un filtro de 3x3

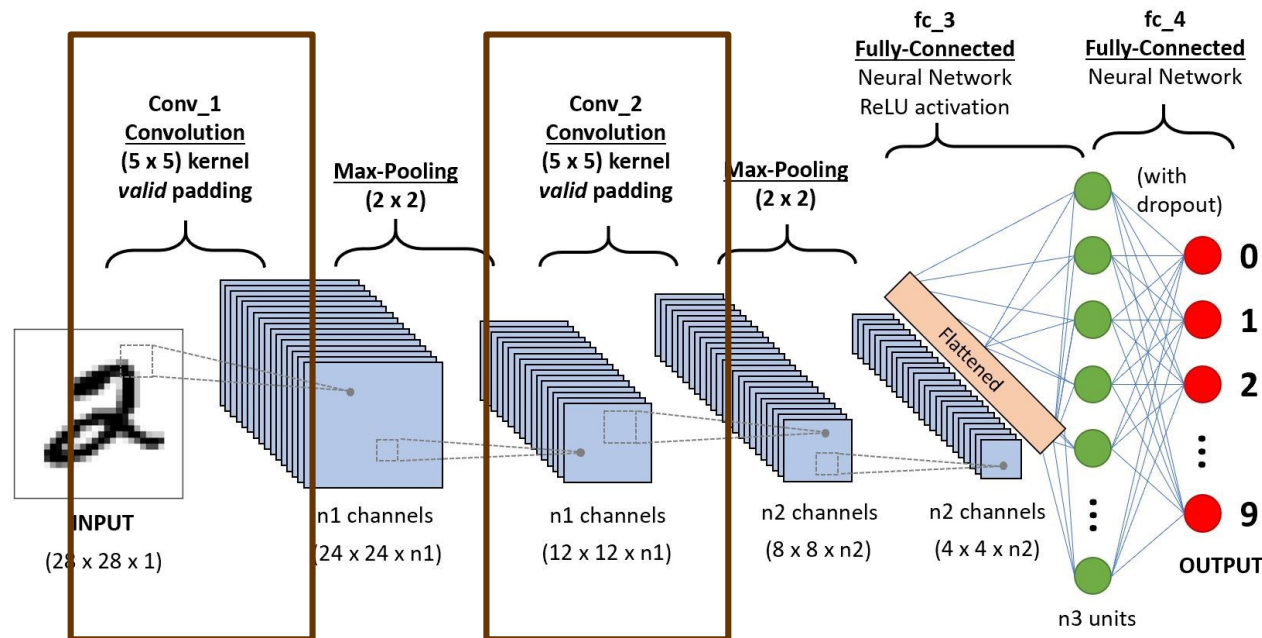


5. Dilatación

Definición

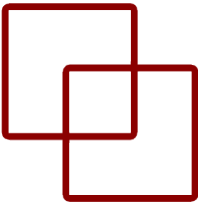


- En los últimos años también se está proponiendo el uso de un parámetro adicional durante la convolución: **factor de dilatación**
- Este parámetro permite desarrollar filtros que puedan trabajar sobre píxeles no consecutivos, buscando la realización de operaciones más ambiciosas, de forma que redes con menos capas logren buenos objetivos

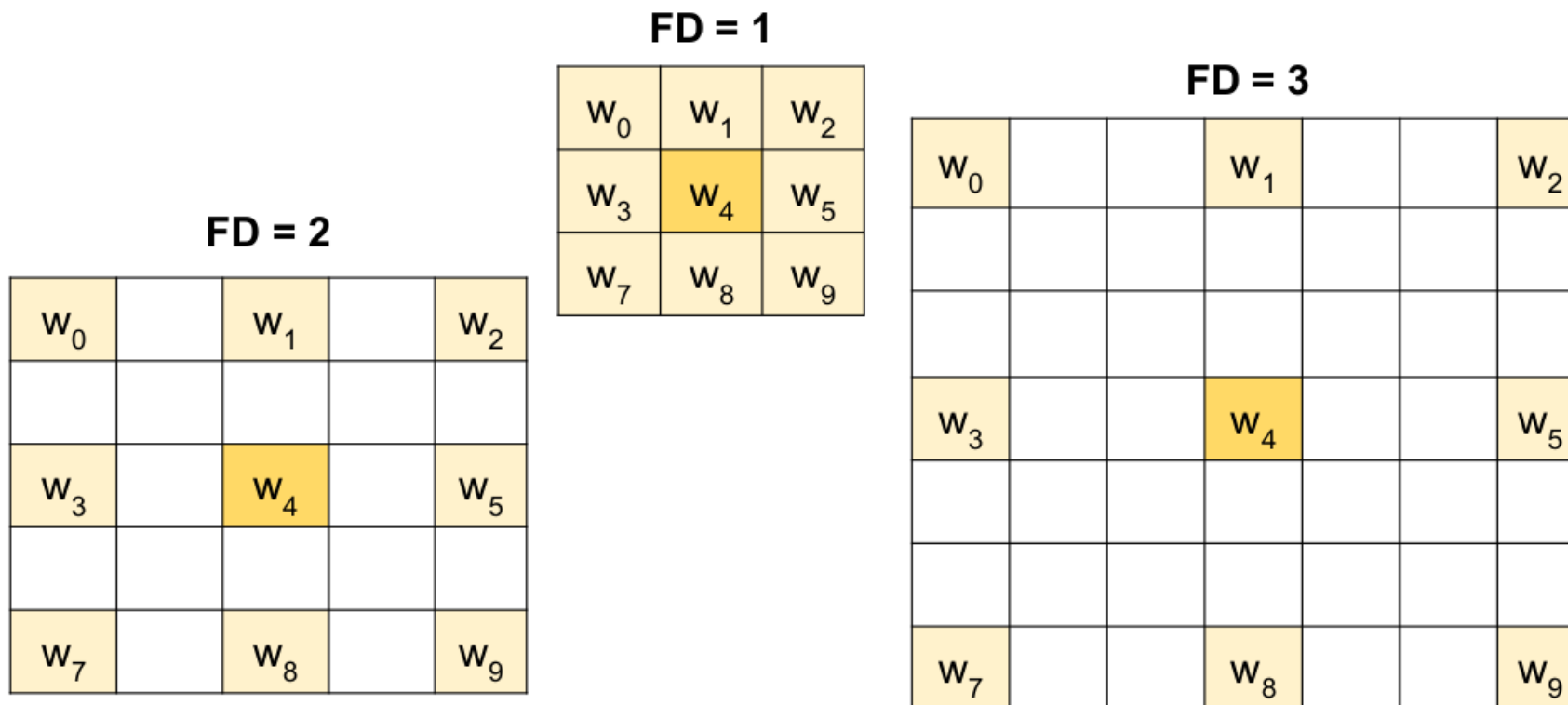


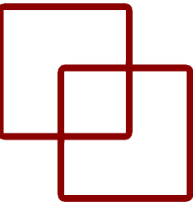
5. Dilatación

Ejemplo



- El factor de dilatación define la separación entre neuronas que se utiliza internamente en el filtro

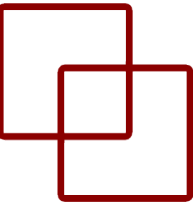




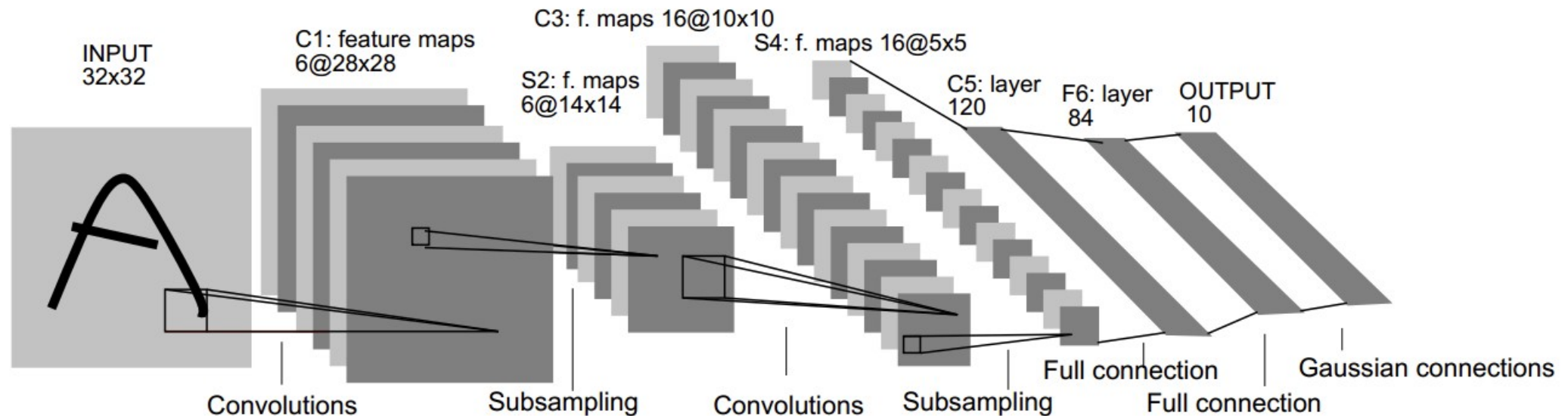
6. Características

- El objetivo de las CNNs no suele ser el aprendizaje de filtros básicos (p.e. bordes), y debemos entender este aprendizaje como un paso necesario (pero no suficiente) para la consecución de metas más complejas
 - Por ejemplo, la detección de bordes puede ser útil para un clasificador de imágenes donde se identifique si la imagen de entrada está codificada o no (pensando en imágenes emitidas por canales de televisión)
- Las CNNs poseen una característica muy interesante:
 - Los filtros (p.e detección de bordes) **no deben ser explícitamente** introducidos en la definición del problema, sino que las características de la red harán que se aprendan de forma **automática**
- Introducimos una de las características clave de las CNNs:
 - Las primeras capas tienden a aprender **características simples** (como bordes), mientras que **características más complejas** (formas básicas) se aprenden a medida que pasamos a capas más profundas.

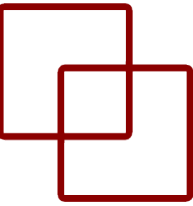
7. Proceso de convolución(II)



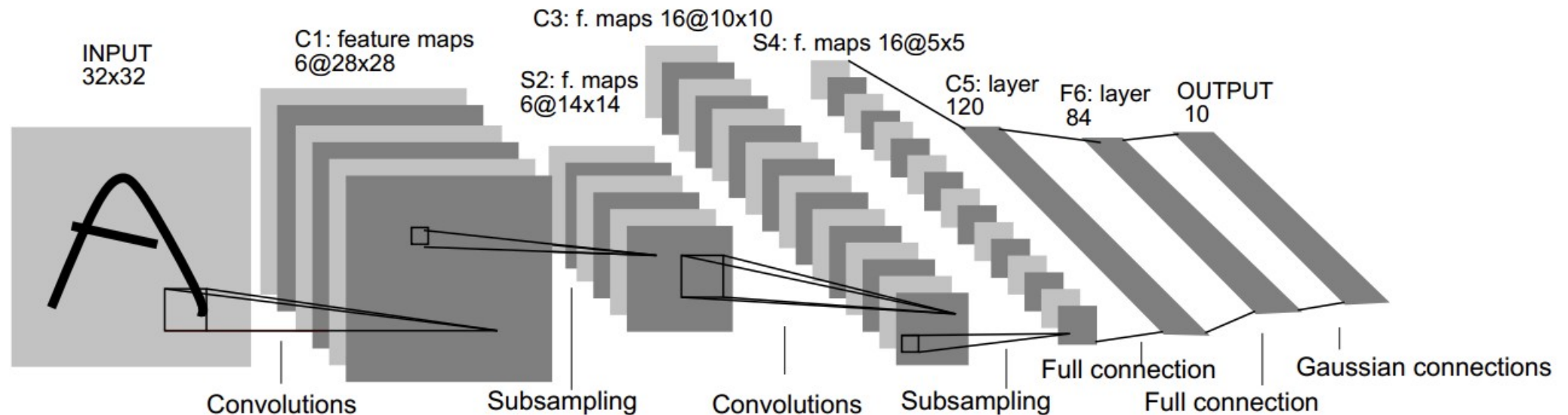
- La operación de convolución no suele aplicarse en solitario: cada fase aplica **N convoluciones paralelas**
- Este proceso genera capas intermedias cuya profundidad no será 1, sino que dependerá de la profundidad de la capa anterior y del número N de convoluciones
- Las dimensiones (ancho y alto) de estas capas vendrán determinadas por la capa anterior, los parámetros *padding* y *stride*, y el tamaño de la convolución

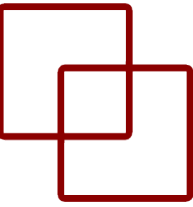


7. Proceso de convolución(I)



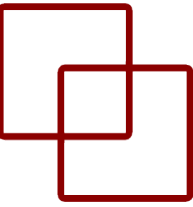
- Estos aspectos hacen que la red **aumente de tamaño** (profundidad) tras cada convolución, lo cual se compensa como veremos a continuación con la operación *pooling* (también conocida como *subsampling*) que permite simplificar.





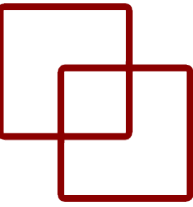
8. Operaciones en convolución

- La operación convolución se desarrolla a través de multiplicaciones de matrices, lo cual favorece una implementación muy **eficiente**



8. Operaciones en convolución

- La operación convolución se desarrolla a través de multiplicaciones de matrices, lo cual favorece una implementación muy **eficiente**
- También simplifica la formulación para la fase de aprendizaje, dado que las funciones que interviene son fácilmente derivables (recordemos *backpropagation* visto anteriormente)

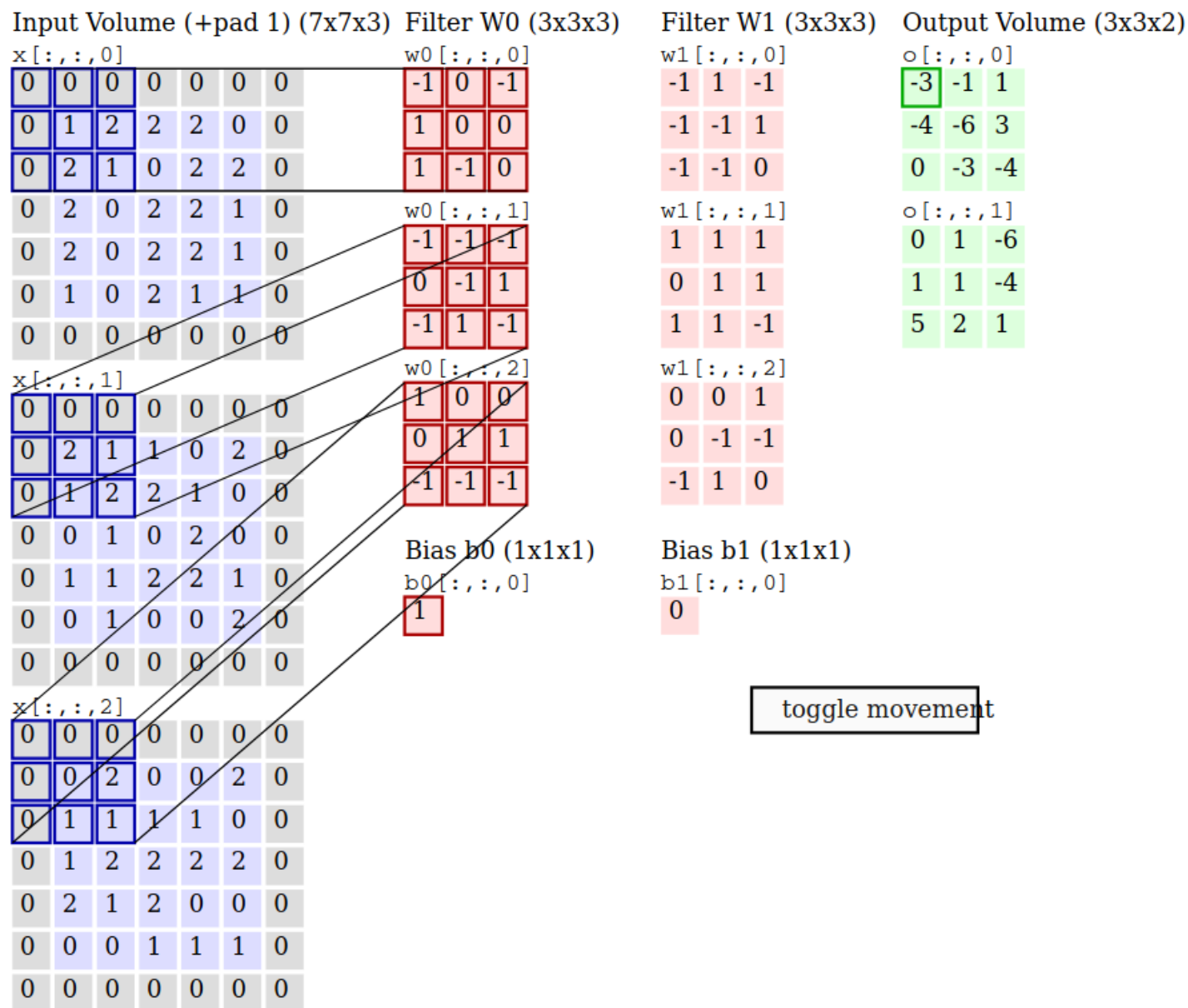
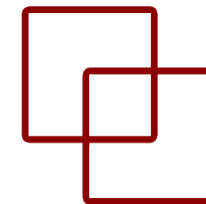


8. Operaciones en convolución

- La operación convolución se desarrolla a través de multiplicaciones de matrices, lo cual favorece una implementación muy **eficiente**
- También simplifica la formulación para la fase de aprendizaje, dado que las funciones que interviene son fácilmente derivables (recordemos *backpropagation* visto anteriormente)
- Ejemplo: <http://cs231n.github.io/convolutional-networks/>

8. Operaciones en convolución

Ejemplo

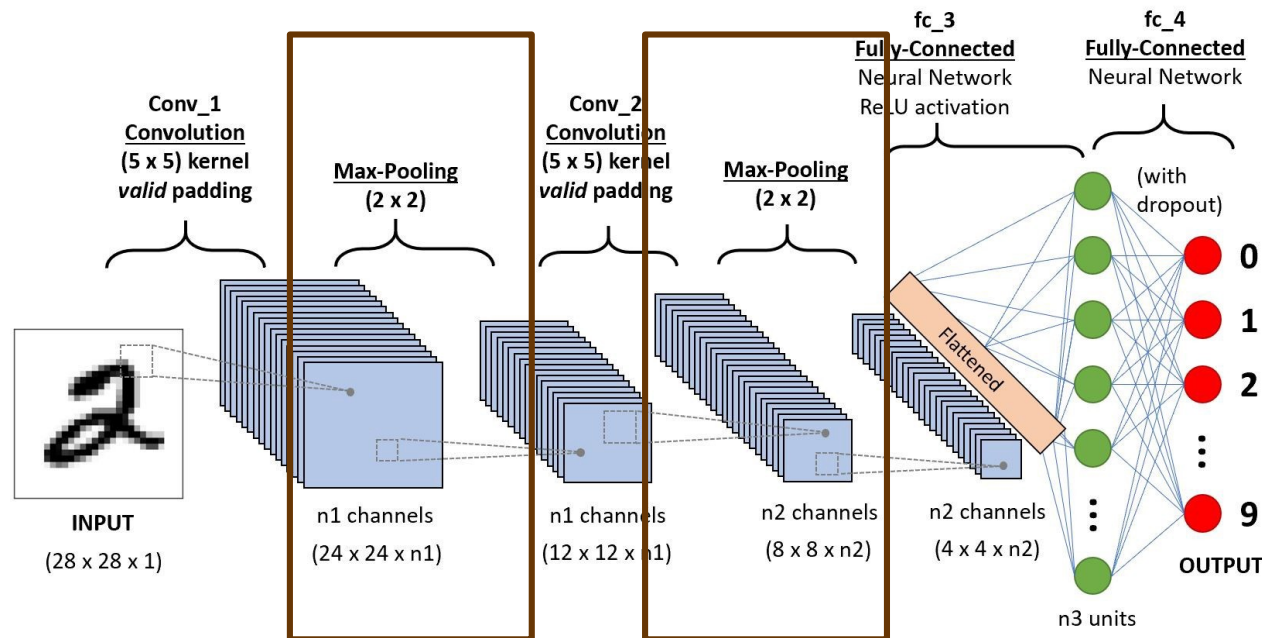


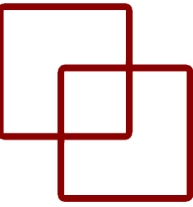


Operación *Pooling*

1. Definición

- La operación *pooling* tiene por objetivo **reducir la dimensionalidad** de las capas generadas tras la aplicación de operaciones de convolución
- Recordando que las capas disponen de tres dimensiones (*ancho x alto x profundidad*), la reducción se aplicaría únicamente sobre las **dimensiones de anchura y altura**, sin afectar a la profundidad de estas capas

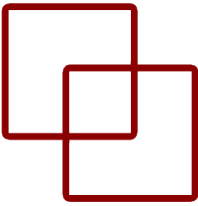




2. Ejemplo

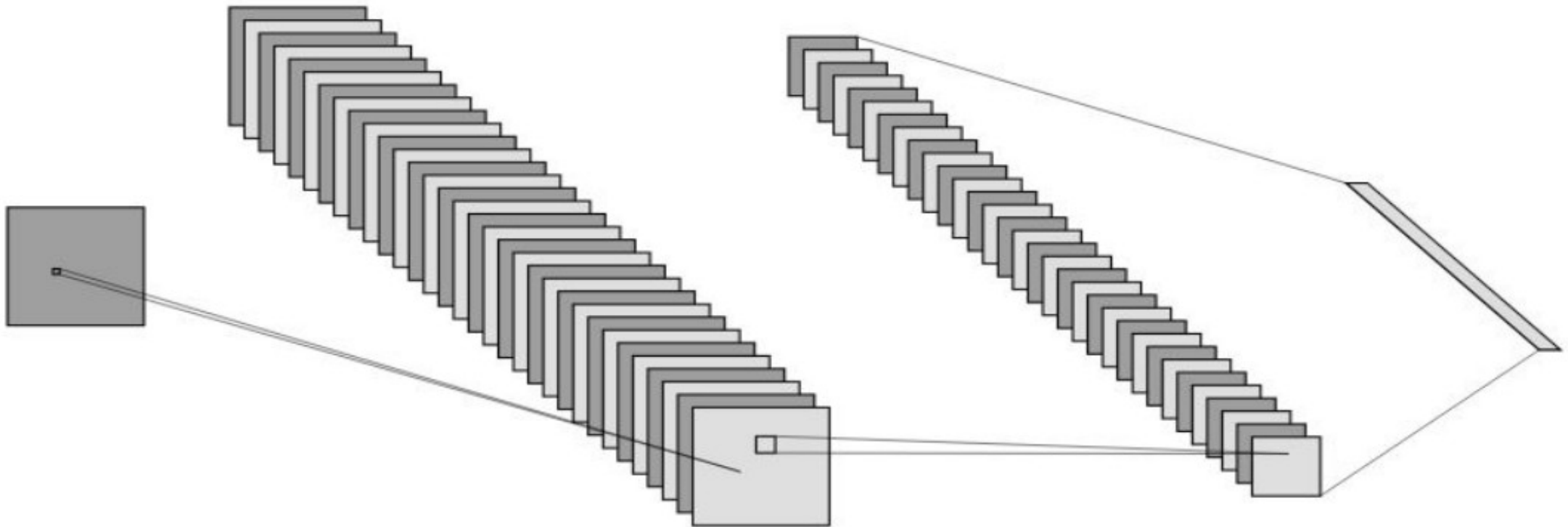
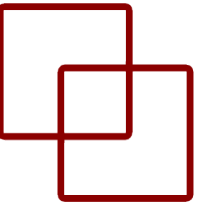
- Sobre un ejemplo simple, asumamos que trabajamos con imágenes **640x480** (B/N) y que la primera convolución aplica **32 filtros de 3x3** (*stride 1, zero-padding 1*)
 - El resultado será una capa intermedia de **640x480x32**

2. Ejemplo

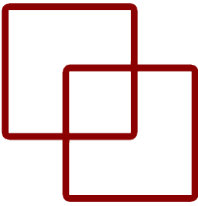


- Sobre un ejemplo simple, asumamos que trabajamos con imágenes **640x480** (B/N) y que la primera convolución aplica **32 filtros de 3x3** (*stride 1, zero-padding 1*)
 - El resultado será una capa intermedia de **640x480x32**
- Una operación *pooling* **reduciendo la dimensionalidad** a un 25% de la original obtendría una nueva capa intermedia de 320x240x32

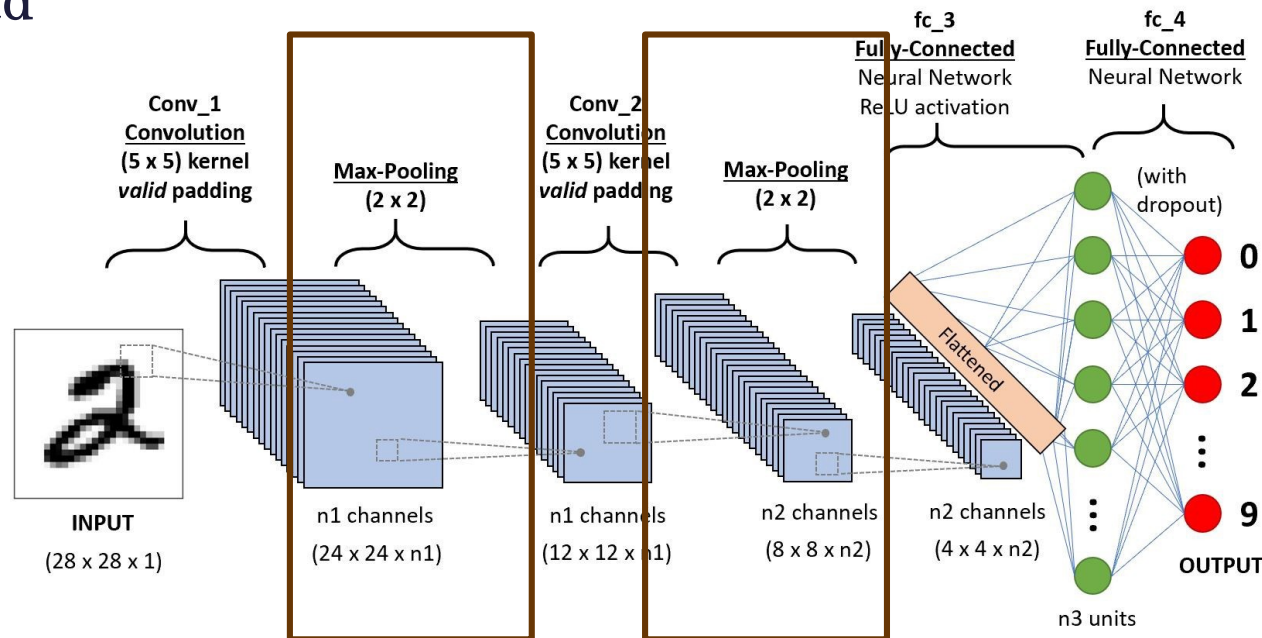
2. Ejemplo



3. Max-pooling

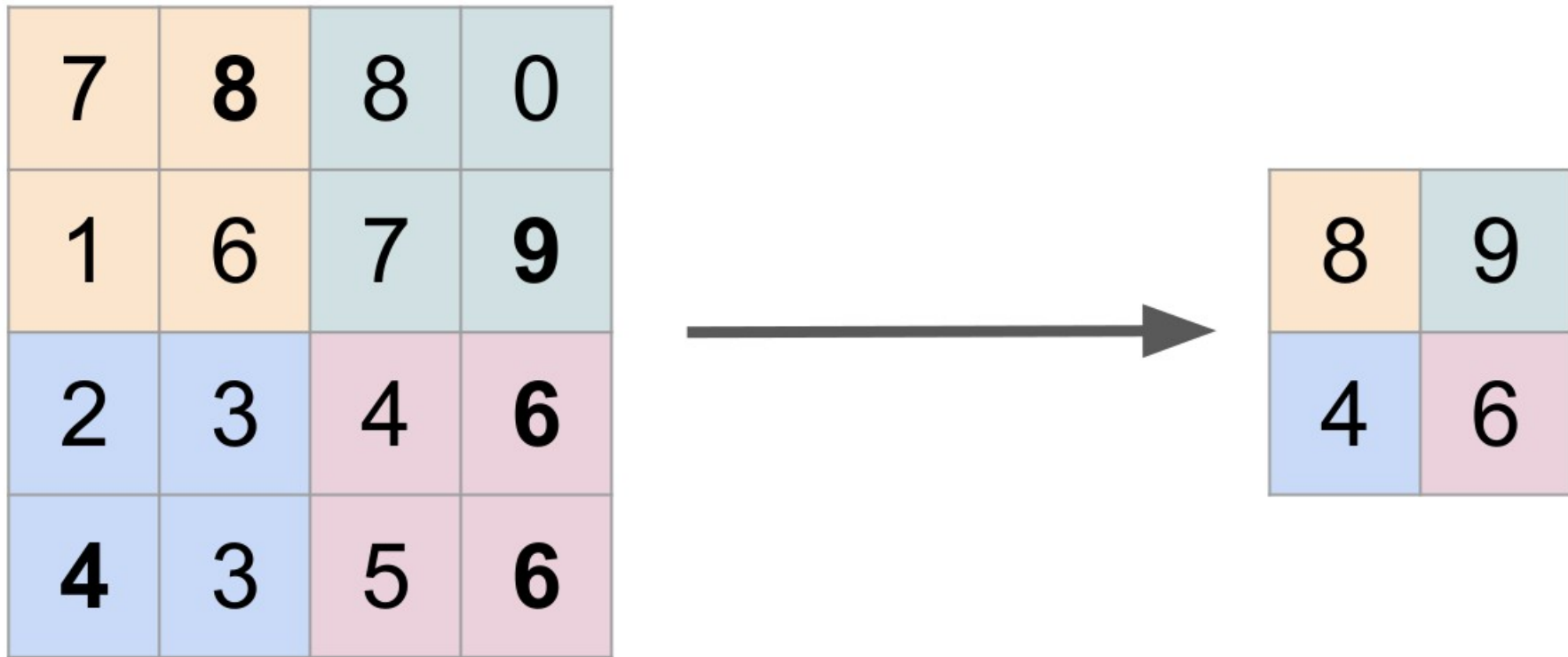
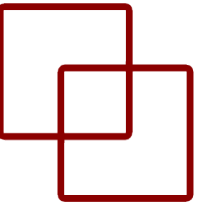


- La operación de *pooling* más común es *max-pooling*, donde cada operación computa el **máximo** para una serie de neuronas **conectadas espacialmente**
- Su aplicación se realiza a través de un filtro de tamaño $N \times M$, normalmente $M=N$ (se define como extensión espacial).
 - También se usa el **parámetro *stride*** que determinará el factor de reducción de dimensionalidad

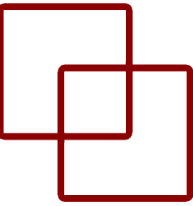


3. Max-pooling

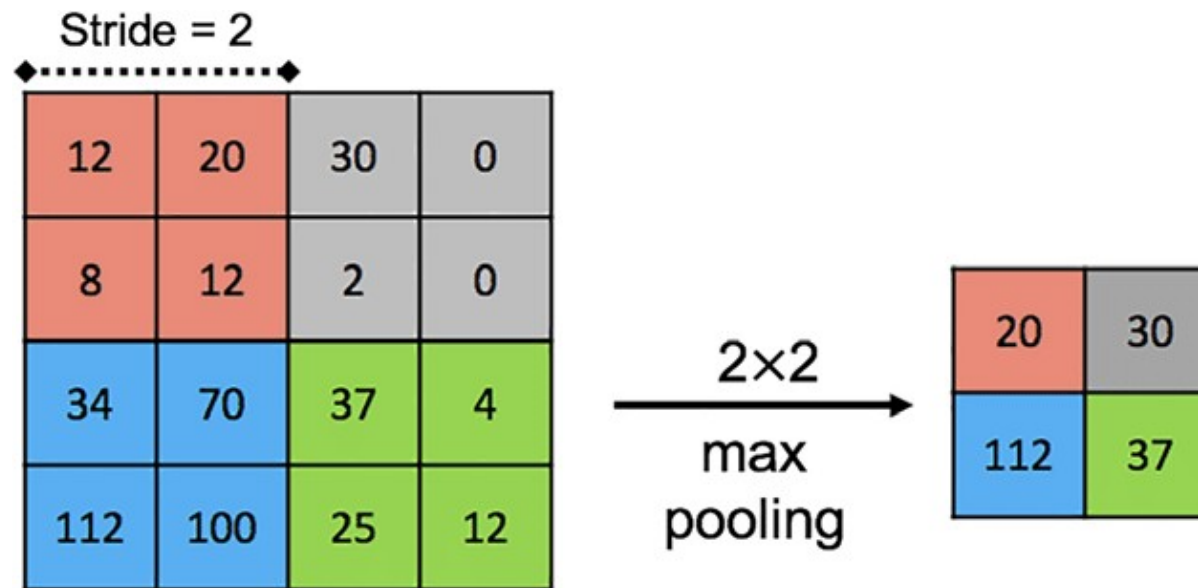
Ejemplo de max-pooling 2x2 con stride=2



4. Consideraciones



- A nivel práctico, el uso de filtros mayores a 3x3, o de un valor de *stride* superior a 2, suele generar malos resultados → desaconsejamos su uso
- Como **alternativas** al *max-pooling*, podemos reemplazar la función máximo por otras funciones como la media.
 - Sin embargo, la experiencia ha mostrado el mayor poder de la función máximo

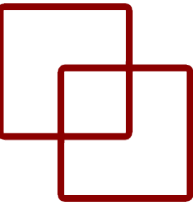




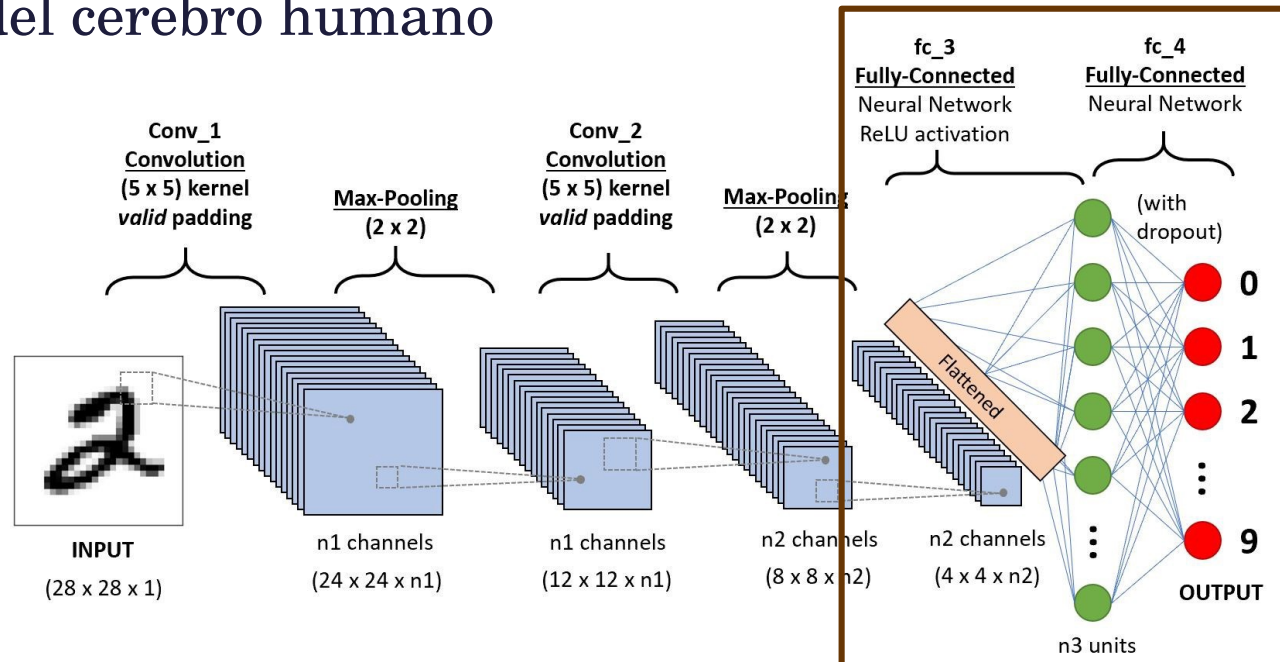
LAB CTIC  UNI

Otros parámetros de la arquitectura CNN

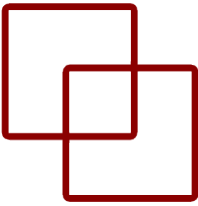
1. Contexto



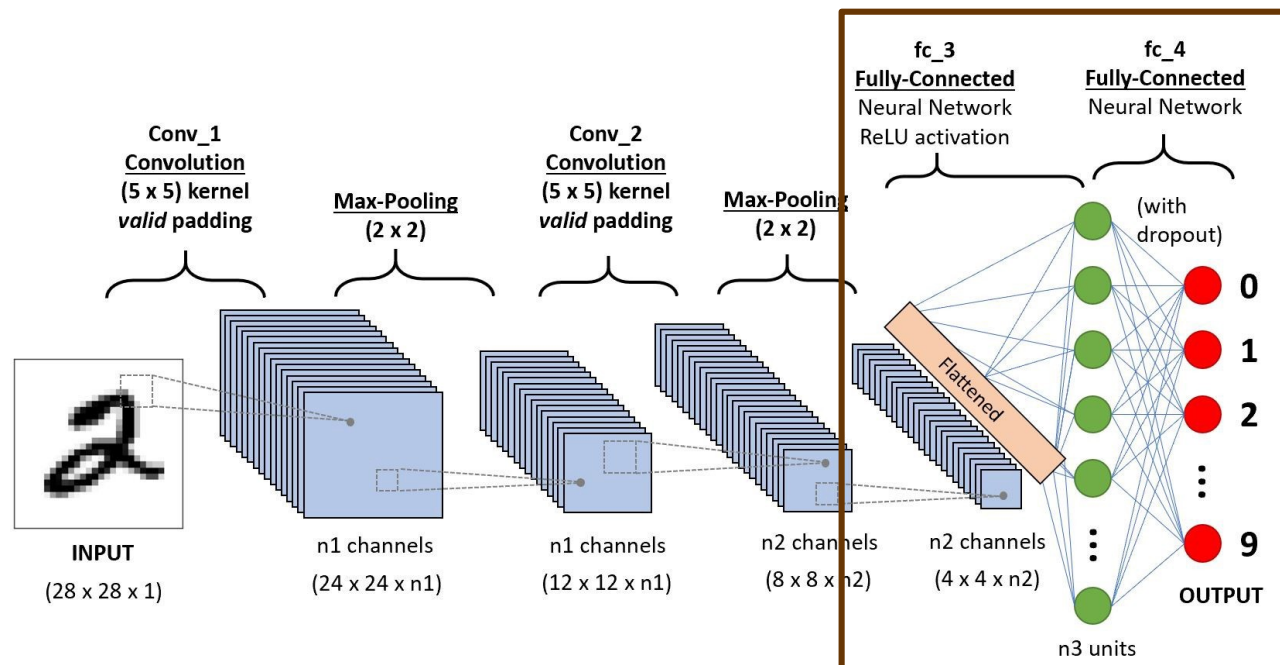
- Tras la combinación de capas de tipo convolución y *pooling*, las etapas finales de una CNN las forman capas de neuronas **totalmente conectadas** (*fully connected*), similares a las capas de las redes neuronales tradicionales
- Aunque han caído en desuso, algunos autores también propusieron el uso de **capas de normalización** siguiendo de nuevo algunos principios de funcionamiento del cerebro humano



2. Funciones finales



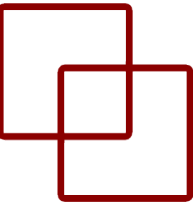
- Las operaciones de convolución suelen incluir una etapa de activación basada en el uso de la función **ReLU**, lo que aporta capacidad de resolución de problemas no lineales (además de las propiedades vistas en la sesión anterior)
- Finalmente, también solemos encontrar una capa **SoftMax** en CNNs enfocadas a la clasificación multiclase, la cual permite interpretar las decisiones del clasificador como probabilidades



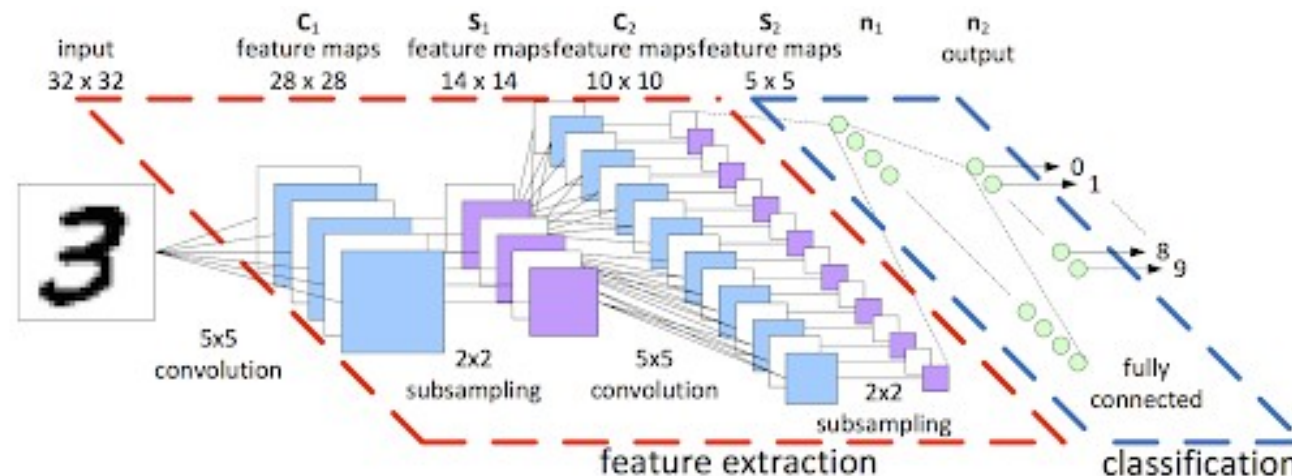


Arquitecturas utilizadas: casos de éxito

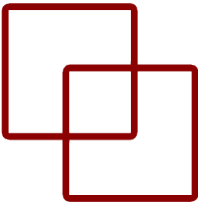
1. Contexto



- En esta última parte del temario vamos a repasar algunas de las **arquitecturas más exitosas** de CNNs, estudiando también la evolución en cuanto a tendencias
- Muchas de estas arquitecturas están implementadas en diferentes soluciones o **frameworks** de Deep Learning, y también es común encontrar modelos ya entrenados sobre las mismas

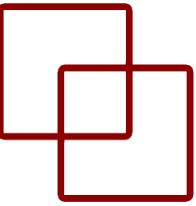


2. Le-Net: Lecun et al. 1998



- 7 capas sin contar la entrada (imágenes de 32x32)
- Capa 1 → Convolución: incluye 6 filtros de 5x5 con stride=1 sin padding (32x32 → 28x28x6)
- Capa 2 → Subsampling: filtros 2x2 (no es max-pooling) con stride=2 (28x28x6 → 14x14x6)
- Capa 3 → Convolución tipo C1 pero con 16 filtros y no conectada totalmente con C2 (14x14x6 → 10x10x16)
- Capa 4 → Subsampling tipo C2 (10x10x16 → 5x5x16)
- Capa 5 → Convolución tipo C1 con 120 filtros que afecta a todas las neuronas de C4 (5x5x16 → 1x1x120)
- Capa 6 → Capa totalmente conectada a C5 con 84 unidades (120 → 84)
- Capa 7 → Clasificación usando 10 clases/categorías y la función de base radial o RBF (84 → 10)

2. Le-Net: Lecun et al. 1998



- Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, november 1998

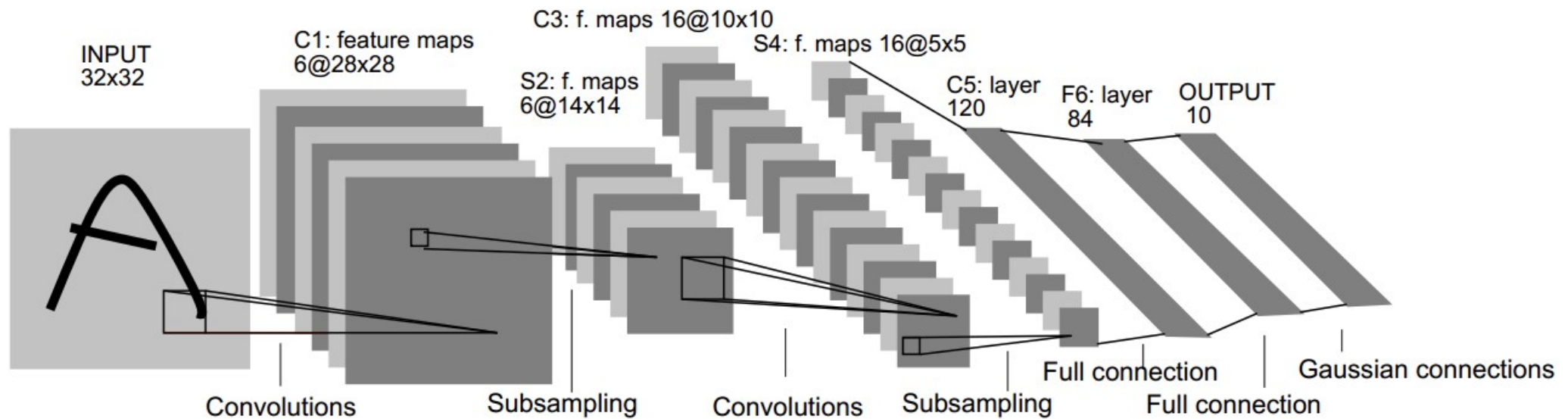
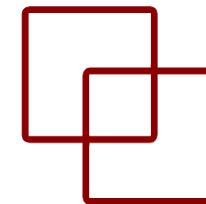
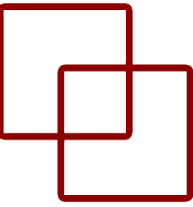


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

3. AlexNet: Krizhevsky et al. 2012

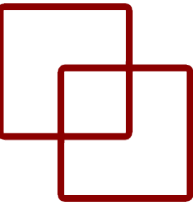


- La entrada son imágenes en RGB de 224x224 píxeles por 3 canales (224x224x3)
- Capa 1 → Convolución con 96 filtros de 11x11 con stride=4 (224x224x3 → 55x55x96)
- Capa 2 → Subsampling: filtros 3x3 (max pooling) con stride=2 (55x55x96 → 27x27x96)
- Capa 3 → Normalización (27x27x96)
- Capa 4 → Convolución con 256 filtros de 5x5 con stride=1 y padding=2 (27x27x96 → 27x27x256)
- Capas 5-6 → Subsampling tipo C2 + normalización (27x27x256 → 13x13x256)
- Capa 7 → Convolución con 384 filtros de 3x3 con stride=1 y padding=1 (13x13x256 → 13x13x384)
- Capa 8 → Convolución con 384 filtros de 3x3 con stride=1 y padding=1 (13x13x384 → 13x13x384)
- Capa 9 → Convolución con 256 filtros de 3x3 con stride=1 y padding=1 (13x13x384 → 13x13x256)
- Capa 10 → Subsampling tipo C2 (13x13x256 → 6x6x256)
- Capas 11-12 → Capas totalmente conectadas con 4096 neuronas (6x6x256 → 4096)
- Capa 13 → Capa totalmente conectada a una salida con 1000 neuronas/clases (4096 → 1000)
- Capa 14 → Softmax



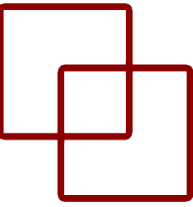
3. AlexNet: Krizhevsky et al. 2012

- Esta red fue la encargada de poner el foco sobre el Deep Learning al ganar la **competición ILSVRC**



3. AlexNet: Krizhevsky et al. 2012

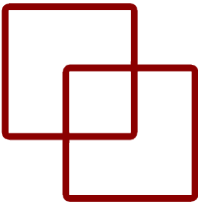
- Esta red fue la encargada de poner el foco sobre el Deep Learning al ganar la **competición ILSVRC**
- Introdujo algunos **principios** que se mantienen hoy en día (uso de ReLU) mientras otros han perdido interés (normalizaciones)



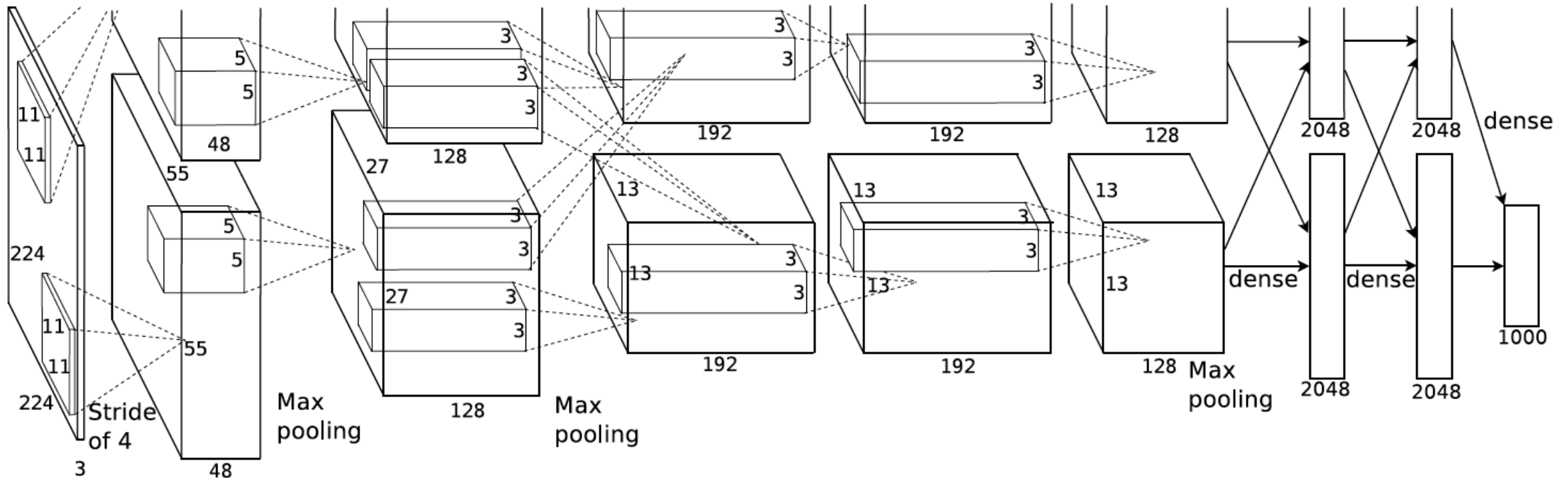
3. AlexNet: Krizhevsky et al. 2012

- Esta red fue la encargada de poner el foco sobre el Deep Learning al ganar la **competición ILSVRC**
- Introdujo algunos **principios** que se mantienen hoy en día (uso de ReLU) mientras otros han perdido interés (normalizaciones)
- Para su entrenamiento se hizo un uso efectivo de **paralelismo**, distribuyendo las operaciones entre dos CPUs

3. AlexNet: Krizhevsky et al. 2012



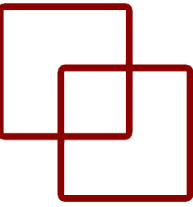
- A. Krizhevsky, I. Sutskever, and G.E. Hinton. *ImageNet Classification with Deep Convolutional Neural Networks*. Advances in neural information processing systems. 2012





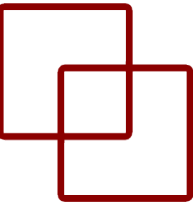
LAB CTIC  UNI

Uso de GPUs para mejora de rendimiento



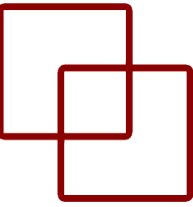
1. Contexto (I)

- Algunos de los datasets disponibles para entrenar redes convolucionales cuentan con millones de imágenes, lo cual se traduce en unos **requisitos de computación** (y memoria) muy elevados



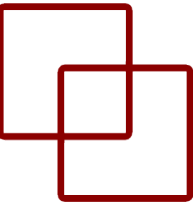
1. Contexto (I)

- Algunos de los datasets disponibles para entrenar redes convolucionales cuentan con millones de imágenes, lo cual se traduce en unos **requisitos de computación** (y memoria) muy elevados
- Para poder entrenar estos modelos de forma efectiva, es muy común el uso de GPUs, cuyo elevado número de núcleos es ideal para realizar **tareas en paralelo**



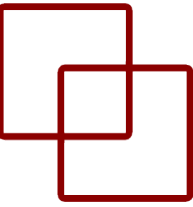
1. Contexto (I)

- Algunos de los datasets disponibles para entrenar redes convolucionales cuentan con millones de imágenes, lo cual se traduce en unos **requisitos de computación** (y memoria) muy elevados
- Para poder entrenar estos modelos de forma efectiva, es muy común el uso de GPUs, cuyo elevado número de núcleos es ideal para realizar **tareas en paralelo**
- Mientras que su uso en otros campos requiere una fase previa de adaptación/optimización del código, el código usado para aprender modelos de Deep Learning incluye numerosas **tareas que pueden realizarse en paralelo**



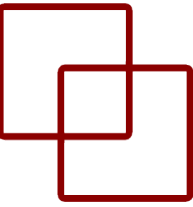
1. Contexto (I)

- Algunos de los datasets disponibles para entrenar redes convolucionales cuentan con millones de imágenes, lo cual se traduce en unos **requisitos de computación** (y memoria) muy elevados
- Para poder entrenar estos modelos de forma efectiva, es muy común el uso de GPUs, cuyo elevado número de núcleos es ideal para realizar **tareas en paralelo**
- Mientras que su uso en otros campos requiere una fase previa de adaptación/optimización del código, el código usado para aprender modelos de Deep Learning incluye numerosas **tareas que pueden realizarse en paralelo**
 - Un ejemplo lo tendríamos en la primera capa de convolución de las CNNs, donde N núcleos permitirían paralelizar el aprendizaje de **N filtros o kernel diferentes**



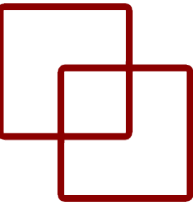
1. Contexto (II)

- Además del número de núcleos, el uso de GPUs en deep learning se utiliza debido a sus capacidades en cuanto a memoria



1. Contexto (II)

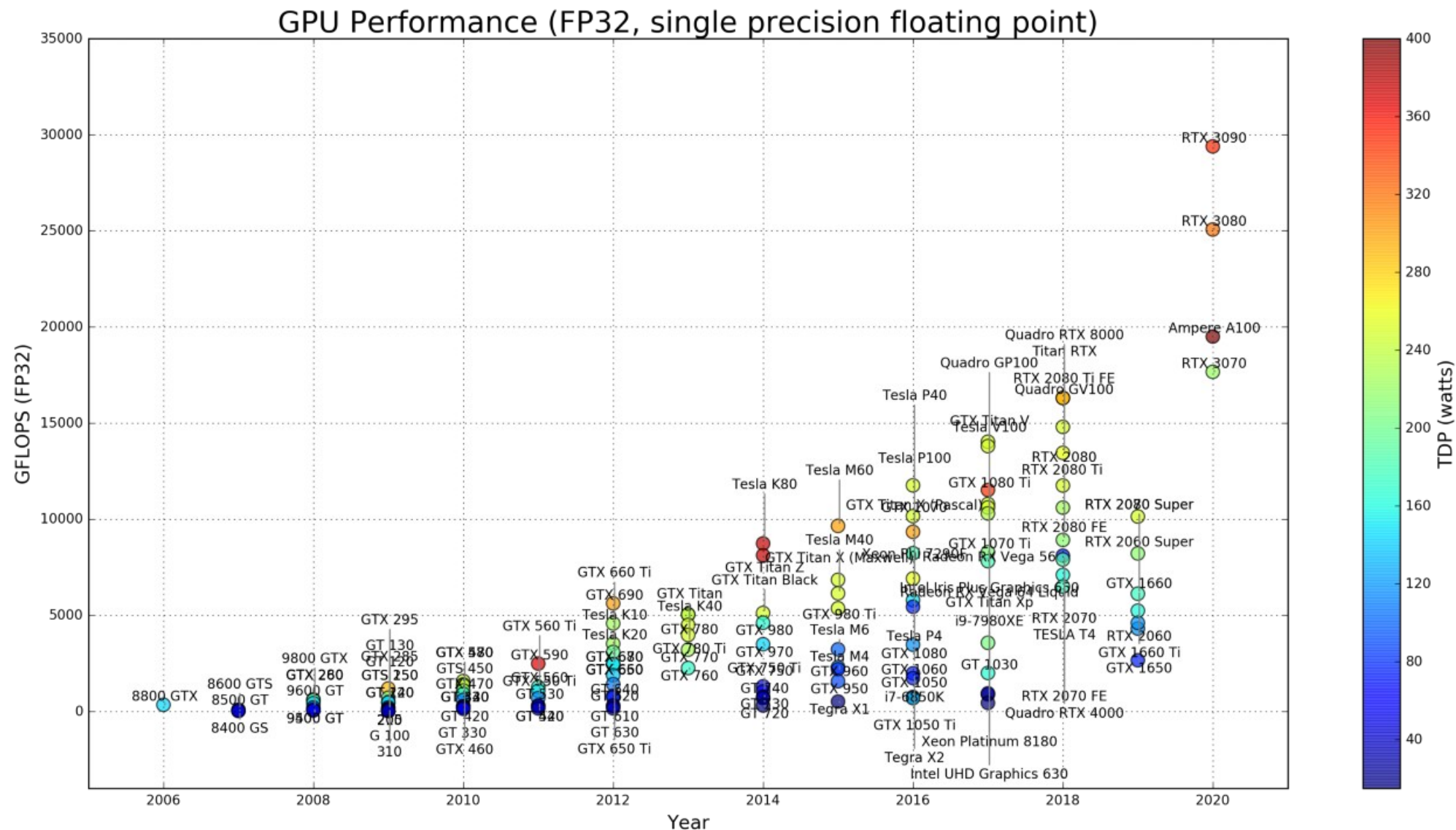
- Además del número de núcleos, el uso de GPUs en deep learning se utiliza debido a sus capacidades en cuanto a memoria
- A nivel de Hardware, Nvidia ha destacado entre el resto de alternativas gracias al *toolkit* CUDA, el cual está integrado en la mayoría de *frameworks* de Deep Learning.



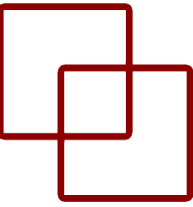
1. Contexto (II)

- Además del número de núcleos, el uso de GPUs en deep learning se utiliza debido a sus capacidades en cuanto a memoria
- A nivel de Hardware, Nvidia ha destacado entre el resto de alternativas gracias al *toolkit* CUDA, el cual está integrado en la mayoría de *frameworks* de Deep Learning.
- A la hora de seleccionar una GPU, lo ideal es buscar la comparativas actuales que evalúen los últimos modelos en términos de rendimiento (y también coste) sobre tareas similares a las que se pretendan realizar

2. Comparativa Hardware



Fuente: <https://blog.inten.to/hardware-for-deep-learning-part-3-gpu-8906c1644664>



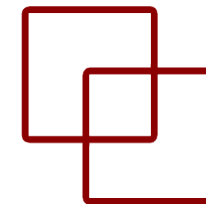
3. Proveedores externos

- En lugar de adquirir Hardware especializado para realizar tareas de Deep Learning, también podemos utilizar los recursos disponibles en los **proveedores de servicios cloud**
- En concreto, podemos hacer uso de instancias equipadas con potentes GPUs (e.g. instancias EC2 tipo P3 en AWS), aunque su coste por hora será elevado
 - Como alternativa para tareas no urgentes, podemos hacer uso de **instancias de tipo *spot***, que pueden costar hasta un 90% menos que las normales

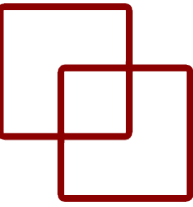


Conclusiones

1. Conclusiones

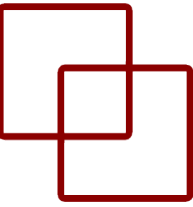


- A lo largo de la sesión, hemos aprendido las **características** y **parámetros** que definen uno de los tipos de redes más populares: CNNs



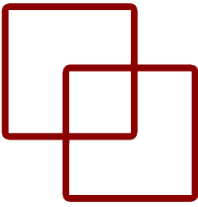
1. Conclusiones

- A lo largo de la sesión, hemos aprendido las **características** y **parámetros** que definen uno de los tipos de redes más populares: CNNs
- El conocimiento de las **operaciones de convolución** y *pooling* es esencial para poder diseñar redes correctamente, dado que debemos tomar decisiones cuya validación puede requerir horas o días



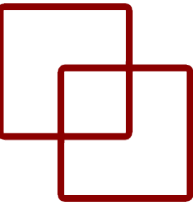
1. Conclusiones

- A lo largo de la sesión, hemos aprendido las **características** y **parámetros** que definen uno de los tipos de redes más populares: CNNs
- El conocimiento de las **operaciones de convolución** y **pooling** es esencial para poder diseñar redes correctamente, dado que debemos tomar decisiones cuya validación puede requerir horas o días
- Para una mejor comprensión del funcionamiento de las CNNs, es conveniente utilizar **representaciones espaciales de las arquitecturas**, de forma que sea fácilmente comprobable la dimensión de alguna de las capas en relación al resto de la red



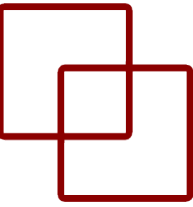
1. Conclusiones

- A lo largo de la sesión, hemos aprendido las **características** y **parámetros** que definen uno de los tipos de redes más populares: CNNs
- El conocimiento de las **operaciones de convolución** y **pooling** es esencial para poder diseñar redes correctamente, dado que debemos tomar decisiones cuya validación puede requerir horas o días
- Para una mejor comprensión del funcionamiento de las CNNs, es conveniente utilizar **representaciones espaciales de las arquitecturas**, de forma que sea fácilmente comprobable la dimensión de alguna de las capas en relación al resto de la red
- Existen numerosas implementaciones para **generar modelos gráficos de CNNs**, pero siempre podemos hacer uso de herramientas online:
 - <http://alexlenail.me/NN-SVG/index.html>
 - <https://github.com/HarisIqbal88/PlotNeuralNet>
 - <https://datascience.stackexchange.com/questions/14899/how-to-draw-deep-learning-network-architecture-diagrams>



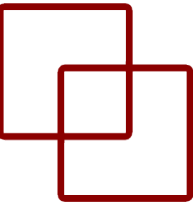
2. Conceptos en prácticas 3 y 4

- Las redes neuronales (y otros modelos de clasificación) tienen un **tamaño de entrada fijo**.
 - Las imágenes son matrices de $N \times M$ píxeles.
 - Para poder clasificarlas todas deben tener las mismas dimensiones.



2. Conceptos en prácticas 3 y 4

- Las redes neuronales (y otros modelos de clasificación) tienen un **tamaño de entrada fijo**.
 - Las imágenes son matrices de $N \times M$ píxeles.
 - Para poder clasificarlas todas deben tener las mismas dimensiones.
- Necesitamos un gran número de imágenes para entrenar una red neuronal (***data augmentation***).



2. Conceptos en prácticas 3 y 4

- Las redes neuronales (y otros modelos de clasificación) tienen un **tamaño de entrada fijo**.
 - Las imágenes son matrices de $N \times M$ píxeles.
 - Para poder clasificarlas todas deben tener las mismas dimensiones.
- Necesitamos un gran número de imágenes para entrenar una red neuronal (*data augmentation*).
- La **arquitectura “general”** de una CNN consta de las siguientes capas:
 - $(\text{CONV} + \text{POOL})^n + \text{FC} + \text{SOFTMAX}$

¡GRACIAS!



Dr. Manuel Castillo-Cara
Deep Learning con Keras y Python
Web: www.smartcityperu.org